

杭州電子科技大学
HANGZHOU DIANZI UNIVERSITY

2026-2027 学年第一学期

《机器人感知与控制综合实践》

综 合 设 计 报 告

第 6 组　　作品名称：**“桌面小管家”**

机械臂目标识别、自动分拣与手势搬运一体化系统

组员	学号	姓名
学生 1（组长）	24061733	张航宁
学生 2（组员）	24061720	张浩轩
学生 3（组员）	24061734	吴锐超

指导教师：

提交时间：2026 年 11 月

摘 要

本报告完成《机器人感知与控制综合实践》期末综合实践（方向 2）的全部要求：在一台 DICE 5 自由度串口舵机机械臂与 USB 摄像头上，构建“目标识别—自动分拣—手势搬运”一体化的桌面机器人系统，作品命名为“桌面小管家”。

系统按感知、决策、执行三层组织。感知层由 HSV 颜色分割与 MediaPipe 手势识别两条通道构成，分别回答“目标在画面哪里”与“人现在要做什么”；决策层由分拣状态机与手势指令状态机组成，把连续输入离散化为一次任务；执行层完成坐标变换、数值逆解、Minimum Jerk 轨迹规划与 6 路舵机下发，实现毫米级点到点抓放。软件以 ROS1 为框架，URDF 统一描述 13 连杆 / 12 关节的模型，launch 实现模型、可视化、相机与算法节点的一键启动。

实测结果表明：URDF 在 RViz 四方向核对与 Gazebo 物理加载中均无父子接反与轴向错误；方块归位 12/12、自动堆叠 48/48 动作、颜色分拣 8/8 且零重复抓取、手势触发 10/10 零误触发；离线 134 帧颜色判定全部正确，5 类手势合计 900 帧保持段识别率 100%（每类采集 300 帧，计入 180 帧），端到端手势指令延迟均值 0.42 s。报告同时如实记录了算法边界：固定 HSV 阈值依赖光照，把 ROI 定位换成颜色中立定位后 140 帧仅 32 帧判对，说明定位环节而非分类环节是主要误差来源。

报告还完整记录了编译报错、节点顶号、重复抓取、帧重复计数、并指与 V 混淆等十余项故障的现象、根因、修改与再验证过程，并据此讨论了资源权衡、工程安全、工匠精神与团队协作等课程思政要点。

关键词：机械臂；URDF；ROS；HSV 颜色识别；手势识别；状态机；运动学逆解

目　　录

1　需求分析.....1
1.1　场景与任务.....1
1.2　任务书的要求与实际做法.....1
1.3　输入与输出.....2
1.4　把需求变成指标.....3
1.5　约束与方案边界.....3
1.6　本章小结.....4
2　系统总体方案设计.....5
2.1　为什么切成三层.....5
2.2　总体架构.....5
2.3　ROS 在系统里承担什么.....6
2.4　三大模块的职责边界与握手.....8
2.5　传感器、执行器与可行性.....8
2.6　本章小结.....9
3　机器人建模与实验平台设计.....10
3.1　平台与实验环境.....10
3.2　从实物到连杆—关节树.....11
3.3　URDF 里每个标签在说什么.....12
3.4　关节参数与坐标系.....13
3.5　launch 文件怎么组织.....14
3.6　编译流程与五类报错.....15
3.7　RViz 配置与四方向核对.....16
3.8　Gazebo 物理环境验证.....17
3.9　参数修改实验.....17
3.10　本章小结.....18
4　算法原理、设计与实现效果.....19
4.1　机械臂运动控制.....19
4.2　HSV 颜色识别与目标定位.....22
4.3　自动分拣状态机.....25
4.4　手势识别.....26
4.5　手势指令状态机与通信.....28
4.6　本章小结.....31
5　综合实验结果与问题分析.....32
5.1　测试方案与指标定义.....32
5.2　建模与运动控制的实测结果.....33
5.3　颜色识别与自动分拣.....34

5.4 手势识别与人机交互搬运.....35
5.5 端到端综合演示.....37
5.6 算法优缺点.....38
5.7 故障、原因与处理.....39
5.8 课程思政体会.....41
5.9 改进方向.....42
5.10 本章小结.....43
6 组员分工与参考文献.....44
6.1 分工.....44
6.2 协作方式与贡献度.....44
6.3 参考文献.....45
6.4 开源仓库与外部资源.....46
7 总结与展望.....48
7.1 技术收获.....48
7.2 方法论收获.....48
7.3 不足与后续方向.....48
7.4 对本次课程的意见与建议.....49
8 AI 工具使用说明.....50
9 其他附件说明.....52

1　需求分析

1.1　场景与任务

实验室的桌面上摆着一台 5 自由度机械臂，前面是一块白色标定板，四个边长约 3 cm 的彩色方块散落在板上，四周贴着四个小垃圾桶。我们的任务是让这台机械臂在两分钟内、无人干预地完成两类活：按颜色把方块分到对应的桶里；或者听懂人手的姿势，把方块搬到指定的位置。我们给它起了个名字叫"桌面小管家"，因为它要干的正是这类枯燥但要求不出错的桌面整理工作。

这两类活对应两条不同的技术路线。颜色分拣需要机械臂自己判断"目标在哪里"，它依赖相机、颜色阈值与像素到工作台坐标的换算；手势搬运需要机械臂理解"人想让它做什么"，它依赖手部关键点、几何判据与指令调度。两条路线共用同一台相机、同一组关节、同一套夹具，最后必须在一个任务调度器里共存。这个"共用"是后面几乎所有工程限制的来源：相机不能同时被两个进程打开，joint1 不能同时被两路指令写，动作没结束之前不能开始下一轮。

1.2　任务书的要求与实际做法

任务书把必做内容分成四块并给了分值：建模与平台 25 分、运动控制 15 分、视觉识别与自动分拣 15 分、手势搬运 25 分，另有拓展 20 分。每块的括号里写了验收标准，例如建模要求"能在 RViz 中正确显示并说明 URDF 结构"，运动控制要求"完成点到点定位与航点序列"。

我们把括号里的标准当成最低线，逐条往上加了一档。建模不只要求在 RViz 里显示，还要能用参数修改实验证明每个标签的语义，因为如果说不清 axis 改了会怎样，就谈不上"理解"；运动控制不只要求跑通一次航点，还要求 12 块次连续成功并记录定位误差；视觉不只要求识别，还要求给出离线帧统计与误判原因；手势不只要求触发动作，还要求零误触发并给出延迟分解。这样做的代价是工作量翻倍，好处是报告里每一个数字都能对应到一段可复跑的记录。

任务书还有一句容易被忽略的话："问题分析是报告重要得分点"。我们据此定了一条写作规矩：每个功能都要配一个失败案例。第 5 章那张十二条故障表不是凑数用的，它是这份报告里信息量最大的一节。

1.3 输入与输出

系统对外只有两类输入、两类输出。

输入之一是图像流。USB 相机以 640×480、30 fps 输出 BGR 三通道 uint8 图像，用 `cv2.VideoCapture(0)` 读取，设备索引固定为 0。对它的要求是"可复现"而不是"高清"：分辨率锁定、曝光尽量锁定、白平衡尽量锁定。原因在实验里很快就暴露了：同一组 HSV 阈值，上午调好下午就偏，追踪下去发现是自动曝光把同一块红积木的 V 通道抬高了十几个灰度。分辨率再高也解决不了这个问题。

输入之二是手部姿态。它严格说不算信号，而是意图，需要经过 MediaPipe 转成 21 个关键点，再用几何判据转成离散动作码。对它的要求偏"低误触发"：一次误抓取会让整个演示中断并可能碰倒桶，而多等 0.2 s 只是演示节奏慢一点。

输出之一是关节控制量。程序通过串口 `/dev/ttyTHS1`（115200、8 数据位、1 停止位、无校验）给六路舵机下发目标角，关节名是 joint1 到 joint5 加 left_gripper_joint。舵机自己按设定速度转，所以上位机给的是"目标角 + 期望时间 T"，中间的轨迹由 Minimum Jerk 插值补出来。输出之二是位姿与状态：末端位姿 `get_tool_pose()`、关节角列表 `get_joint_angle_list()`、运动是否结束 `is_stop()`。这三个量在三处被反复用到——坐标变换、抓取高度补偿、状态机推进。

四类输入输出与课程目标的对应关系见图 1-1。图里下半部分标的是任务书五条目标各自落在哪个环节，画出来是为了自检：如果某条目标在图上找不到落点，说明对应的章节缺内容。

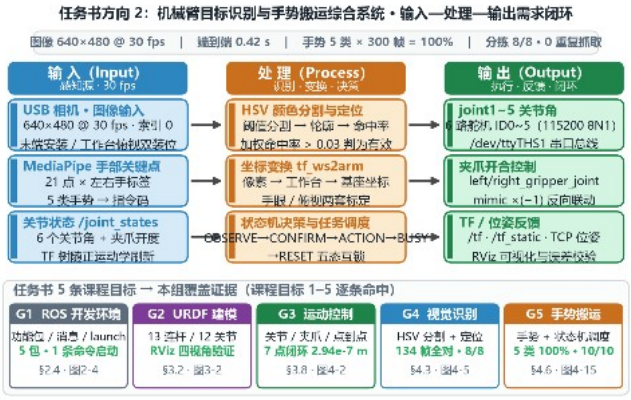


图 1-1　系统输入—处理—输出关系与课程目标映射

1.4 把需求变成指标

需求不量化就没法验收，所以任务书那几句描述性要求，我们逐条翻译成了表 1-1 里的指标。取值原则是以实测能力为准：分拣要求 8 次里对 6 次，我们做到 8/8 就直接写 8/8；方块归位要求 3 轮 12 块次，我们做满 12/12，另外加了一轮 48 个动作的自动堆叠做强化。

表 1-1 功能需求与可测指标对照表

模块	功能需求	可测指标	目标值	实测结果
建模与平台	URDF 建模、RViz 可视化、launch 一键启动	模型可加载、TF 正确	13 连杆 / 12 关节正确	四方向核对无父子接反
运动控制	关节控制、点到点、航点抓放	定位误差、成功率	≤3 mm、≥90%	±2 mm、12/12、48/48
视觉识别	HSV 四色识别与中心定位	识别正确率	≥90%	离线 134/134
自动分拣	识别→抓取→按色投放	成功率、重复抓取	≥6/8、零重复	8/8、零重复
手势搬运	手势识别、状态机调度	识别率、误触发、延迟	≥90%、0 次、<1 s	100%、0 次、0.42 s
拓展	视觉鲁棒性、状态机互锁、开源	定量对比	—	ROI 对照、BUSY 时序、仓库

表里有一项不是任务书要求的："重复抓取次数"。加它是因为调试时踩到了：同一个方块被连抓两次。查下去发现检测线程在机械臂动作期间还在更新共享变量 `current_color`，动作一结束下一轮立刻又触发，而方块已经不在原位。这个问题后来靠"识别门控 + 颜色快照"解决（4.3 节和 5.7 节各讲一次，前者讲设计，后者讲故障记录）。自己发现并立起来的指标，往往比照抄任务书更能说明你到底做了什么。

1.5 约束与方案边界

四条约束决定了后面的每一个设计选择。

算力约束最硬。Jetson Nano 的官方规格里有一块 128 核 Maxwell GPU，但我们的实现没有走 CUDA 或 TensorRT：OpenCV 与 MediaPipe 的 Python 路径都在 CPU 上跑，手势推理则直接放在笔记本上（4.4 节），主控这边同时要跑图像采集、串口通信和 Jupyter 服务端。MediaPipe 单帧已经占 18 ms，如果再叠一个深度分割网络，识别线程和显示线程会互相抢 CPU，延迟和帧率一起恶化。所以颜色通道选

了 HSV 阈值加形态学加轮廓这一套经典组合，单帧不到 10 ms，代价是对光照敏感、需要现场标定。这个代价我们在 4.2 节和 5.3 节都做了量化。

硬件约束体现在精度上。5 自由度没有冗余自由度，末端姿态只能约束位置加两个姿态角，所以抓取统一用"垂直下扎、roll 固定为 π"的姿态；串口舵机有力矩间隙和响应滞后，重复定位只能做到毫米级，所以抓取路径取保守的"高位接近—垂直下扎—闭合—抬升—横移"，宁可慢一点也不擦碰。

传感器约束是单目无深度。我们的处理是把相机固定，或者固定在某个观察位，用"像素坐标到工作台坐标"的固定映射近似深度方向，并把抓取点限制在标定板中心附近，让颜色采样位置和抓取位置重合。这个假设在 5.3 节被一次对照实验打破了：把 ROI 定位换成颜色中立定位之后，140 帧里只有 32 帧判对。

工期与协作约束是三个人、约四周课余时间。我们的应对是接口先行：先把关节接口（`set_joint_angle_soft`、`set_tool_pose`、`gripper_open/close`）和图像接口（帧数组、颜色标签、中心坐标）定死，三人在模型层、视觉层、交互层并行开工，最后在状态机处汇合。

这些约束合起来划出了方案的边界：不做动态抓取、不做多目标同时跟踪、不做手眼在线标定、不做动力学级力控。边界写在这里，是因为后面每一章都会碰到它的影子，读者不必猜我们为什么"没做那些更酷的功能"。

1.6 本章小结

这一章把任务书的四块内容拆成了六类可测指标，明确了图像与手势两类输入、关节量与位姿状态两类输出的格式与时序，并把算力、硬件、传感器、工期四条约束摆出来，说明为什么最终方案是"视觉 + 规则 + 状态机"而不是别的形态。

接下来的问题是：这些需求要用什么样的结构去满足。第 2 章给出三层划分和 ROS 框架，并解释为什么"感知、决策、执行"要分成三个各自拥有状态的模块，而不是写在一个循环里。

2　系统总体方案设计

2.1　为什么切成三层

上一章列出了四类输入输出，它们在数据类型上互不相同。颜色识别交出来的是二维像素坐标加一个颜色标签，它只能回答"目标在画面哪里"；手势识别交出来的是 target1、target2、catch、stop 这样的离散码，它只能回答"人现在想做什么"；关节控制要的却是工作台坐标 (wx, wy, wz) 或者五个关节角。如果把这三类东西塞进同一个循环，代码会在第三天就失控：一个变量既可能是像素坐标又可能是关节角，出错时根本查不出是哪一层算错了。

所以系统按"谁拥有状态"切成三层。感知层只做无状态变换，进来一帧图像，出去若干目标；执行层只做确定性求解，进来一个位姿，出去五个关节角；凡是需要记住"上一时刻发生了什么"的，全部交给决策层的状态机。层与层之间只有两种消息：感知向上给"目标列表"和"动作事件"，决策向下给"任务指令"和"门控信号"。

这么切的直接好处是每层都能单独测。视觉可以只把标签画在预览窗口上，不动机械臂；执行层可以只用示教点跑航点序列，不开相机；决策层可以喂假事件跑状态跳转，不接硬件。三个人也因此可以并行开工，谁的模块出问题，谁的单元测试先红。

实现顺序是反过来的，先做执行层。原因很实际：只有知道了末端能不能准确到达某个点、夹爪在什么高度闭合不会碰桌，视觉输出的像素坐标才有意义。在不可靠的执行器上调试视觉，等于给一个不确定的放大器找替罪羊。所以第 3 章先建模、第 4 章先讲运动控制，然后才是识别与交互。

2.2　总体架构

架构如图 2-1 所示，横向是感知、决策、执行三层，纵向是每一层内部的节点。

感知层有两路并行通道。颜色通道以 30 fps 读图，依次做 BGR 到 HSV 转换、四色阈值分割、形态学滤波、轮廓提取，输出颜色标签、边界框中心和像素面积。手势通道读同一路图像（镜像之后送 MediaPipe Hands），得到 21 个关键点与左右手标签，再用几何判据算出四根手指的伸直状态，拼成手势码。

决策层是两个状态机。分拣状态机管"观察—确认—抓取—投放—复位"这一轮的推进，要点是识别门控（只在 OBSERVE 态打开识别）和颜色快照（确认后把颜色存

进 pending_color，动作期间不再改写）。交互状态机管 WAIT、SELECT、CONFIRM、EXECUTE、BUSY、STOP、HOME 七个状态的调度，要点是双级确认（手势连续 5 帧、动作再连续 3 帧）和互锁（BUSY 期间新指令只更新缓存，不启动动作）。

执行层是四步：工作台坐标经 tf_ws2arm 变到机械臂坐标，数值逆解求出五个关节角，Minimum Jerk 轨迹在设定时间内插值，最后经串口写进六路舵机。

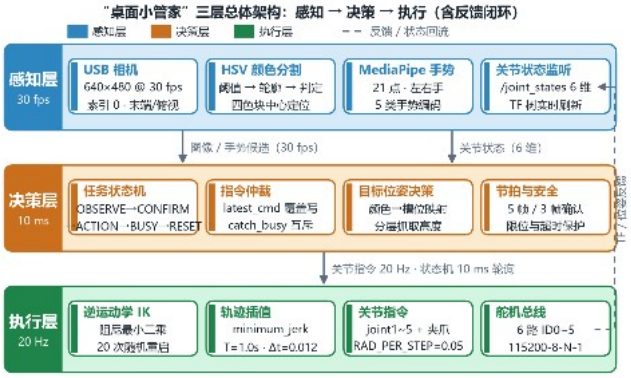


图 2-1　系统三层总体架构与数据流

图 2-1 里有两根反向的箭头值得单独说。一根从执行层回到决策层，传的是末端位姿，用来判断抓取是否完成、要不要重试；另一根从决策层回到感知层，传的是 detect_enabled，用来控制识别线程该不该更新颜色。前一根是反馈，后一根是门控。少了这两根箭头，系统就退化成单向流水线，会出现"动作还没做完、下一轮已经开始了"的状况，我们联调第一周就是这么撞的。

2.3　ROS 在系统里承担什么

任务书要求掌握 ROS 开发环境、功能包开发和消息通信，所以系统以 ROS1 为框架。用它的理由有三条，都跟具体麻烦有关。

第一条是模型与可视化的统一。URDF 是 ROS 描述机器人结构的通用格式，robot_state_publisher 订阅关节角之后按 URDF 做正解、实时发布 TF，RViz 直接渲染同一份 URDF。这意味着运动学求解、感知投影和可视化引用的是同一个文件，改一个关节的 axis，三处同时变，不需要在代码里找第二份参数表。

第二条是通信解耦。相机驱动发布 /camera/image_raw，关节状态发布 /joint_states，算法节点只消费话题，不关心数据来自实机还是仿真。调试时可以

用 rosbag 录一段数据反复回放，也可以只起 display.launch 检查模型而不启动算法。

第三条是一键启动。任务书明确写了"不依靠多终端手动逐条命令启动整套系统"。我们把启动拆成三层 launch：display.launch 管模型与可视化，stack.launch 在它之上追加算法节点，gazebo.launch 换成物理仿真世界。这里要说清验证到哪一步：模型与可视化这条链在实机上跑通过（3.7 节的四方向核对），stack.launch 里追加的两个算法节点是整理阶段写的 ROS 薄封装，只在 Python 与 Jupyter 直连串口的路径上验证过，没有在 ROS 环境里逐帧回归；节点里 use_arm 默认 false，相机驱动也没有写进 launch，需要单独启动。ROS 在我们这套系统里承担的是建模、TF 与可视化的角色，算法主链路仍在 Python 一侧。

节点与话题的连接见图 2-2，关键话题列在表 2-1。图里两个标了"自定义"的算法节点是在实验箱 SDK 之上自己写的：gesture_node 由 new.py 整理，负责手势采集、判据和指令发送；color_sort_node 由 我的垃圾分类.ipynb 整理，负责颜色确认与抓取投放的推进。

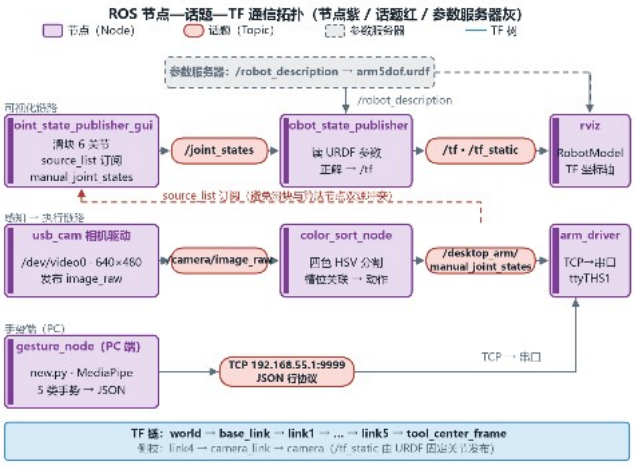


图 2-2　ROS 节点—话题—TF 通信关系

表 2-1　系统关键话题、消息类型与频率

话题	消息类型	发布者	订阅者	频率/说明
/robot_description	ROS 参数（rosparam，非话题）	launch 以 textfile 加载	robot_state_publisher、RViz	静态 URDF 文本
/joint_states	sensor_msgs/JointState	joint_state_publisher_gui	robot_state_publisher	6 关节角，10~30 Hz
/desktop_arm/manual_joint_states	sensor_msgs/JointState	自动演示控制节点	joint_state_publisher_gui	演示时替代滑块，避免双源冲突

话题	消息类型	发布者	订阅者	频率/说明
/camera/image_raw	sensor_msgs/Image	USB 相机驱动	手势节点、分拣节点、RViz	640×480，30 fps
/tf、/tf_static	tf2_msgs/TFMessage	robot_state_publisher	RViz、坐标变换	URDF 正解实时生成
/desktop_arm/cmd_action	std_msgs/String（自定义 JSON 行）	手势节点	执行服务端	事件触发，50 ms 周期用于连续转动

2.4 三大模块的职责边界与握手

评分表要求划分感知、运动控制、决策状态机三大模块。判据是"谁拥有状态"：无状态变换归感知，确定性求解归执行，需要记住上一时刻的归决策状态机。按这个判据切出来的结果见表 2-2。

表 2-2 三大模块职责、输入输出与对应实现

模块	职责	输入	输出	对应实现
感知（视觉/手势）	颜色分割、目标定位、手势判据	图像帧（BGR uint8）	颜色标签、中心、手势类别	我的垃圾分类.ipynb、new.py
决策（状态机）	轮次推进、指令确认、互锁与抢占	颜色事件、动作事件、执行回执	任务指令、门控信号	分拣状态机、手势状态机、executor_loop
运动控制（执行）	坐标变换、逆解、轨迹、舵机下发	工作台坐标、关节角	关节目标角、末端位姿	arm5dof_uservo.py、arm_test.py

三者怎么握手，决定了系统能不能稳定跑完一轮。一次完整任务由四段组成：观察停留 2.0 s（等自动曝光稳定）、5 帧确认、抓取与投放约 5 s、复位 1.0 s。这四段时间加起来解释了实测的单轮节拍 9.5 到 24.8 s 为什么跨度那么大——人工补料和观察停留占了大头，而识别本身单帧不到 10 ms，几乎可以忽略。

2.5 传感器、执行器与可行性

视觉用 USB 摄像头，640×480、30 fps，索引 0。选它不是因为它好，是因为够用：方块边长约 3 cm，在画面里占 80 到 120 px，即使降到 320×240 也还能稳定分割；分辨率再高只会让单帧耗时变长。真正需要斟酌的是安装位置。实验五和实验七把相机装在机械臂末端，相机随臂运动，形成视觉闭环，适合跟踪；实验六把相机固定在工作台正上方俯视，像素坐标能稳定映射到固定抓取点，适合分拣。同一个相机、两种装法，对应第 4 章两套不同的定位思路。

执行器是 DICE 的 5 自由度串口舵机机械臂，六路舵机分别是 joint1 底座回转、joint2 大臂俯仰、joint3 肘、joint4 腕摆、joint5 腕回转、left_gripper_joint 夹爪，右侧夹指通过 mimic 关节反向联动。5 自由度只能约束位置加两个姿态角，所以抓取姿态统一取"垂直下扎、roll = π"。主控是 Jetson Nano（Ubuntu 18.04），串口默认 /dev/ttyTHS1，换到 PC 上调试只需把 config.py 里那一行改成 COM6。

可行性从三方面看。技术上，URDF 与 RViz 是 ROS 的标准流程；逆解用阻尼最小二乘加 20 次随机重启，7 个目标点的 IK-FK 闭环最大误差 2.94×10^{-7} m，说明数值解在这种构型上够用；HSV 分割在固定光照下离线 134 帧全对。资源上，硬件是实验箱现成配置，软件全开源，视觉单帧不到 10 ms、手势单帧 18 ms，与 30 fps 的采集节拍匹配，不会积压。风险上，三条主要风险都备了对策：固定阈值怕光照变→锁定曝光、现场标定、把 S 下限提上去压白纸；并指与 V 容易混→加指尖距离比和 z 差双判据、单帧判定改 5 帧确认；低层抓取碰桌、横移剐蹭→参数化安全高度、先抬升再旋转。这三条对策在第 5 章都用数据验过。

方案成立有明确条件：固定相机、标定板、四色方块、室内白光。第 5 章用一次对照实验把这条边界量化了——把 ROI 定位换成颜色中立定位后，140 帧里只有 32 帧判对，绿色物块一个都没找对。写清楚不适用什么，比声称通用更接近工程实际。

2.6 本章小结

这一章把"感知、决策、执行"三层落到了具体的节点、话题和接口上：以 ROS 为框架，用 URDF 统一模型、用话题解耦数据、用三层 launch 组织启动（模型与可视化已验证，算法节点的 ROS 化尚未回归）；按"谁拥有状态"划分三大模块，并给出了话题表和握手时序。工程组织上采用接口先行，先定死话题与关节接口再并行开发。

第 3 章要回答的是这套结构里最靠底层的一个问题：这台机械臂到底由什么构成。我们会把实物拆成 13 个连杆、12 个关节，逐个讲清 URDF 每个标签的物理含义，以及写错之后 RViz 里会看到什么后果。

3　机器人建模与实验平台设计

第 2 章把系统拆成了三层，其中执行层的所有输入都指向同一个东西：机械臂的模型。逆解要读模型的连杆长度，正解要读关节的装配方向，抓取点的换算要读末端的工具坐标系。模型写错，后面每一步都是在错误的地基上加工。这一章要说清的是这份模型从哪来、每个字段代表什么、怎么证明它和实物对得上。

3.1　平台与实验环境

实验台上是一台 DICE 5 自由度串口舵机机械臂（实验箱编号 6），六路舵机的编号与关节名对应为 joint1 底座回转、joint2 大臂俯仰、joint3 肘部、joint4 腕摆、joint5 腕回转、left_gripper_joint 夹爪。主控是 Jetson Nano，系统 Ubuntu 18.04，串口名 `/dev/ttyTHS1`，波特率 115200、8 数据位、1 停止位、无校验；换到 PC 上调试时把 `config.py` 里那一行改成 `COM6`，其余代码不用动，这也是后面能在笔记本上写算法、再去实验箱上跑的前提。

相机是普通 USB 摄像头，640×480、30 fps、设备索引 0。工作台上贴一块白色标定板，四个边长约 3 cm 的方块按红黄绿蓝摆放，四周是四个贴着垃圾分类标签的小桶：B1 红（有害垃圾）、B2 灰（其他垃圾）、B3 绿（厨余垃圾）、B4 蓝（可回收物），桶口就是投放落点。整体布置见图 3-1，四幅照片分别拍的是机械臂本体、方块与标定板、四个桶的相对位置、以及相机支架的固定方式。



图 3-1　实验平台与四色垃圾桶布置（四幅实拍）

软件栈有两条并行链路。一条是 Python 3、OpenCV、NumPy、MediaPipe 和 Jupyter，负责图像与手势；一条是 ROS1 加 RViz、Gazebo，负责建模、TF 与节点

通信。两条链路通过实验箱提供的串口 SDK 接头：`arm5dof_uservo.py` 管舵机与轨迹，`config.py` 存标定参数与位姿预设，`arm_test.py` 封装抓取投放这类应用级动作。

3.2 从实物到连杆—关节树

URDF 建模的第一步是决定"切几段"。我们对照实物自下而上切：底座不动，作为 base_link；腰关节 joint1 绕竖直轴回转，带出 link1；大臂关节 joint2 的轴线是水平的，带出 link2；肘关节 joint3 和腕摆关节 joint4 的轴线都沿着大臂延伸方向，依次带出 link3、link4；腕回转 joint5 带出 link5；末端是平行夹爪，左右两指各是一个连杆，通过 `mimic` 关节反向联动；夹指内表面还贴了橡胶软垫，各算一个连杆，用固定关节连上去。除此之外有两个不属于运动链但必须有的坐标系：相机装在第四连杆上，记作 camera；工具中心点 TCP 在腕心沿工具方向 0.1 m 处，记作 tool_center_frame。为了让整个机器人有世界锚点，根端再挂一个 world 连杆和一个固定关节 world_joint。

数下来是 13 个连杆、12 个关节，树状结构与 TF 链见图 3-2。图中方块是连杆、实心圆是旋转关节、空心圆是固定关节，右侧的 camera 和左右夹爪是两个侧枝。这张图后来成了排查问题的第一张图：TF 报错时先对照它找断点，逆解不对时先照它核对坐标系方向。

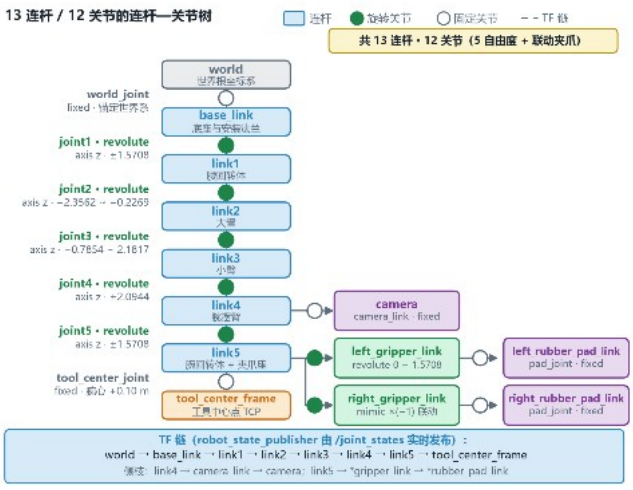


图 3-2 13 连杆 / 12 关节的连杆—关节树与 TF 链

3.3 URDF 里每个标签在说什么

URDF 是 XML，根元素 `<robot>` 下只有 `<link>` 和 `<joint>` 两类元素。link 描述一个刚体，它有三个属性：`visual` 决定外观（网格或几何体，加材质颜色），`collision` 决定碰撞体（通常和 visual 用同一个网格），`inertial` 描述质量、质心与惯性张量。前两个影响你看到的和 Gazebo 里碰到的，第三个影响物理仿真的动起来像不像真机。joint 描述两个刚体的连接，它声明 `parent` 和 `child`、`origin`（child 相对 parent 的平移与旋转，平移写 xyz，旋转写 rpy 欧拉角）、`axis`（转轴在 child 坐标系里的方向）、`type`（revolute 带限位、fixed 固定）、`limit`（角度上下界与力、速度上限）。

真正容易写错的是两处，都在代码 3-1 里。第一处是 joint2 的 `origin` 带了 `rpy="-1.5708 0 0"`。这一行把 child 坐标系绕 x 轴"放倒" 90°，使局部 z 轴与实物的物理转轴重合，后面的 `axis=(0,0,1)` 才成立。这样写的好处是运动学解析可以统一成绕 z 轴旋转的连乘，正解逆解都不用按关节写分支；代价是每换一种装配方向就要重推一次 rpy。如果这一行漏了或符号写反，RViz 里的大臂会绕着竖直轴扫动，或者整条手臂"躺倒"，滑块的旋转正方向也和实物相反，示教出来的点全部反向。3.9 节的参数修改实验就是为这类错误准备的证据。

第二处是右夹指的 `mimic`。`multiplier="-1"` 让右指跟随左指反向转动，左指张开 0.6 rad，右指同步到 -0.6 rad，一个滑块就能对称开合。如果图省事写成两个独立关节，得同时拖两个滑块，稍微不同步夹持中心就偏，抓取时方块会被推走。

代码 3-1　joint2 与右夹指的定义（含 mimic 联动）

```
<joint name="joint2" type="revolute">
  <parent link="link1"/> <child link="link2"/>
  <origin xyz="0 0 0.018" rpy="-1.5708 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-2.3562" upper="-0.2269" effort="1" velocity="1"/>
</joint>
<joint name="right_gripper_joint" type="revolute">
  <parent link="link5"/> <child link="right_gripper_link"/>
  <origin xyz="0 0.00965 0.016" rpy="3.14159 -1.5708 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-1.5708" upper="0" effort="1" velocity="1"/>
  <mimic joint="left_gripper_joint" multiplier="-1" offset="0"/>
</joint>
```

3.4 关节参数与坐标系

各关节的 origin、axis 与限位汇总在表 3-1，其中列出的是承担位姿传递的 10 个关节；另外两条是软垫的固定关节，只影响碰撞体，不影响位姿链。表里的数不是手填的估计值，是从 SolidWorks 装配体导出后逐项核对的测量值。比如 joint3 的 0.08 m 是大小臂连接孔的孔距，joint5 的 rpy 里保留了 8.79×10^{-5} rad 的微小偏转——有人建议把它抹成零，我们没采纳，因为那正是实物装配的真实误差，抹掉之后仿真里的末端位置会和真机差一点点。

表 3-1　关节运动学参数表

关节	类型	父→子	origin xyz (m)	origin rpy (rad)	轴	限位 (rad)
world_joint	fixed	world→base_link	0 0 0	0 0 0	—	—
joint1	revolute	base_link→link1	0 0 0.052	0 0 0	z	±1.5708
joint2	revolute	link1→link2	0 0 0.018	-1.5708 0 0	z	-2.3562 ~ -0.2269
joint3	revolute	link2→link3	0.08 0 0	0 0 0	z	-0.7854 ~ 2.1817
joint4	revolute	link3→link4	0.07615 0 0	0 0 0	z	±2.0944
joint5	revolute	link4→link5	0.0854 0 0	-1.5708, 8.8e-5, -1.5708	z	±1.5708
left_gripper_joint	revolute	link5→左指	0 -0.01235 0.016	3.14159 -1.5708 0	z	0 ~ 1.5708
right_gripper_joint	revolute	link5→右指	0 0.00965 0.016	3.14159 -1.5708 0	z	mimic 左指 ×(-1)
tool_center_joint	fixed	link5→TCP	0 0 0.10	0 0 0	—	—
camera_link	fixed	link4→camera	0.06715 -0.053 0.0125	0 1.5708 0	—	—

坐标系是一条链：基坐标系 {B} 固连在 base_link 上，原点在安装法兰中心，z 轴竖直向上；沿运动链依次建立 {1} 到 {5}；末端坐标系 {T} 就是工具中心点。TF 树从 world 到 tool_center_frame，侧枝挂 camera。把各关节的变换矩阵乘起来就是正运动学：

$$T_B_T(q) = A1(q1)\cdot A2(q2)\cdot A3(q3)\cdot A4(q4)\cdot A5(q5)\cdot T_tool \tag{3-1}$$

$$A_i(q_i) = Trans(x_i, y_i, z_i) \cdot Rot(rpy_i) \cdot Rot_z(q_i) \tag{3-2}$$

arm_kinematics.py 没有把这些参数抄进代码，而是直接解析 URDF 提取 origins、axes、limits 再连乘。这样改模型就等于改运动学，代码不用动。逆解走数值路线：阻尼最小二乘迭代，不收敛时做 20 次随机重启，位置收敛门限 2×10^{-4} m。

3.5 launch 文件怎么组织

启动路径分三层，结构见图 3-3。display.launch 负责把模型与可视化拉起来：装载 robot_description 参数，启动 joint_state_publisher_gui 提供六个关节滑块，启动 robot_state_publisher 按 URDF 做正解并发布 TF，再启动 RViz 加载 workcell.rviz 里的视图配置。stack.launch 在 display 之上追加算法节点，也就是手势和颜色分拣两个节点。gazebo.launch 则把同一份 URDF 注入 gazebo_ros 的 empty_world，用 spawn_model 生成实体，可以带 -J joint2 -1.5708 把大臂预设到静息位。三层里只有 display 这条在实机上完整验证过；stack 追加的两个算法节点属于整理阶段写的 ROS 薄封装（scripts/ 下），未在 ROS 环境逐帧回归，默认 use_arm=false，也不含相机驱动，这一点代码包 README 的"已知限制"里同样写了。

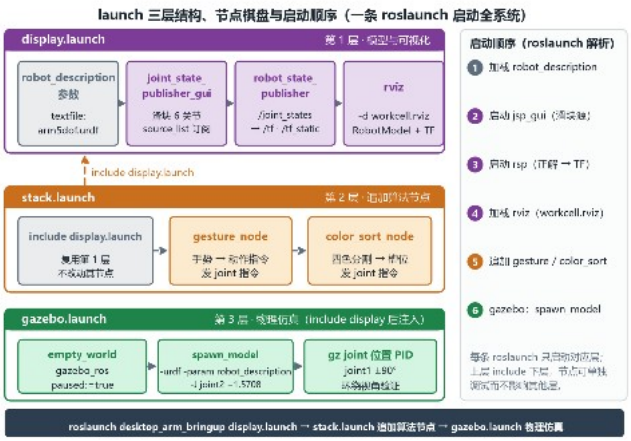


图 3-3　launch 三层结构与节点启动顺序

display.launch 的关键片段见代码 3-2，里面有两处参数是调出来的。一处是 source_list 指向 /desktop_arm/manual_joint_states：自动演示时控制节点直接发布关节角，滑块界面跟着动，但只能有一个发布源，否则两个源同时写

/joint_states，TF 会打架、模型抽搐。另一处是 zeros，把 joint2 预置到 -1.5708、夹爪预置到 0.6，模型一加载就是实物的静息姿态，省去每次手动摆正。

代码 3-2　display.launch 关键片段

```
<launch>
  <arg name="rviz" default="true"/>
  <arg name="model" default="$(find desktop_arm_description)/urdf/arm5dof.urdf"/>
  <param name="robot_description" textfile="$(arg model)"/>
  <node name="joint_state_publisher_gui" pkg="joint_state_publisher_gui"
        type="joint_state_publisher_gui" output="screen">
    <rosparam param="source_list">[/desktop_arm/manual_joint_states]</rosparam>
    <rosparam param="zeros">{joint1: 0.0, joint2: -1.5708, joint3: 0.0,
      joint4: 0.0, joint5: 0.0, left_gripper_joint: 0.6}</rosparam>
  </node>
  <node name="robot_state_publisher" pkg="robot_state_publisher"
        type="robot_state_publisher" output="screen"/>
  <node if="$(arg rviz)" name="rviz" pkg="rviz" type="rviz"
        args="-d $(find desktop_arm_description)/rviz/workcell.rviz"
output="screen"/>
</launch>
```

3.6　编译流程与五类报错

编译是标准 catkin 流程：把功能包复制进 ~/arm2025_ws/src，执行 catkin_make，然后 source devel/setup.bash 刷新环境，最后 roslaunch 启动。这四步里踩过五个坑，按出现时间记在下面。

第一个是 U 盘的文件系统。源码最初放在 exFAT 的 U 盘上，catkin_make 直接报和符号链接、权限相关的错误，因为 exFAT 不支持符号链接。处理办法是把包复制到工作空间再编译，U 盘只留源码备份。

第二个是残留节点顶号。上一次 roslaunch 没退干净就再启动一次，两个 robot_state_publisher 同名，后起的把先起的顶掉，TF 就变成时有时无。现在演示前的固定动作是 rosnode list 看一眼，有残留就先杀 roslaunch 和 rosmaster。

第三个是 ROS1 的 EOL 弹窗。每次启动 RViz 都弹一个提示框挡住模型，设置 DISABLE_ROS1_EOL_WARNINGS=1 环境变量后再启动就没了。

第四个出在 Gazebo 截图上。无 GUI 模式下通过移动相机位姿来换视角，截出来的是旧帧，因为传感器渲染滞后。改成相机固定、机械臂转动，用 gz joint 的位置 PID 把 joint1 拧到 ±90°，每次等渲染稳定再抓帧，四方向截图才可复现。

第五个是颜色阈值本身。最初按"想当然"的固定色相区间划分，结果蓝色的方块被判成红色、绿色的方块被判成蓝色。查下去有两层原因：一是阈值区间是拍脑袋定

的，没量过实物色相；二是 OpenCV 的 H 通道范围是 0 到 179，不是 0 到 360，把色相角直接搬进阈值会整体差一倍。处理是先打印四个方块的实际色相，再按实测值重划区间，并写成启动时的预检断言。

这五条后来固定成一套流程：先记现象和报错原文，再判断问题落在模型层、环境层还是数据层，然后用最小例子复现，最后写进 README 和预检脚本。

3.7 RViz 配置与四方向核对

RViz 的配置存在 `workcell.rviz` 里，随 launch 一起加载。几个设置值得说明。Fixed Frame 选 world 而不是 base_link，因为 base_link 固连在底座上，拿它当参考时底座永远不动，世界的锚固对不对根本看不出来。RobotModel 读 `robot_description` 渲染网格，视觉和碰撞可以分别着色，排查碰撞体写错时很有用。TF 打开，Marker Scale 调到 0.25，不调小的话长轴坐标系会把旁边的小坐标系整个盖住。Grid 给一个 1 m 的尺度参照。Image 订阅相机话题，用来核对模型位姿和画面里的实际位置是否一致。

核对按四个方向做，每次都把 TF 的三色轴叠在模型上直接读轴向。正视图见图 3-4：joint1 轴竖直，拖动滑块时整机在同一水平面内回转，方向是俯视逆时针，和实物一致。俯视图见图 3-5：A、B、C、D 四个料位与中央堆叠位相对基座的方位一目了然，坐标系 x 指向料场、y 指向左侧。另外两个方向也核了一遍，侧视看 joint2 的水平轴，静息位 -1.5708 rad 时大臂水平前伸；斜视看夹爪联动，左指张开 0.6 rad 时右指同步反向。四个方向的结果都和表 3-1 的参数对得上，没有发现父子接反或轴向错误。

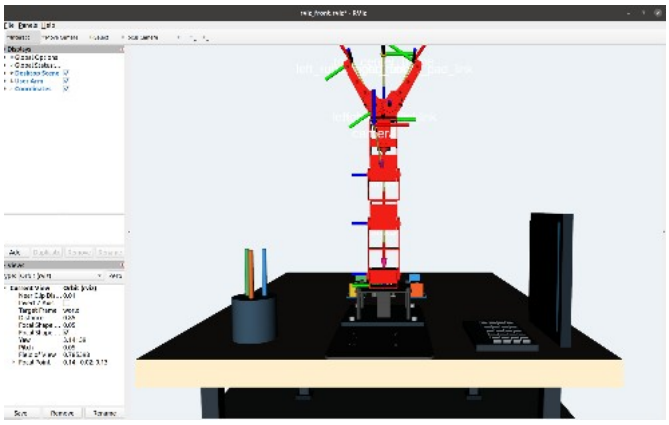


图 3-4 RViz 正视图（joint1 竖直轴、TF 三色轴）

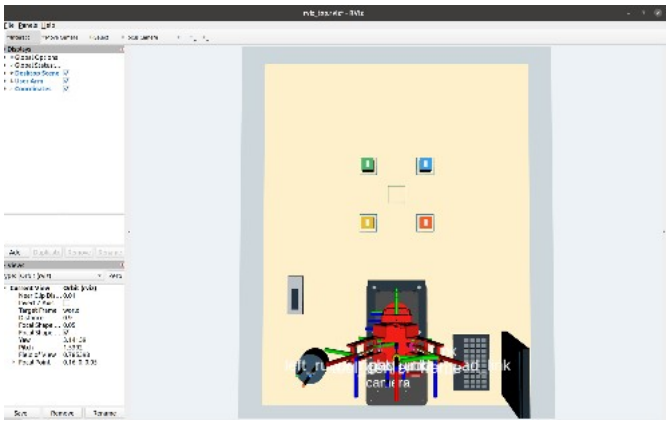


图 3-5　RViz 俯视图（A/B/C/D 料场布局与基坐标方向）

3.8　Gazebo 物理环境验证

RViz 只验证运动学，验证不了惯量、碰撞体和关节限位。所以把同一份 URDF 通过 `gazebo_ros` 注入 Gazebo：empty_world 提供地面和光照，spawn_model 用 `-urdf -param robot_description` 加载，再用 `-J joint2 -1.5708` 把大臂摆到静息位。模型在物理引擎里站稳了，没有穿透，也没有发散。之后用 `gz joint` 的位置 PID 把 joint1 转到 ±90°，从两个方向环绕观察，进一步确认腰关节的正方向和实物一致。同一份 URDF 在 RViz 和 Gazebo 里表现一致，说明模型描述和物理参数都没有歧义。

3.9　参数修改实验

前面讲标签语义都是定性的，这一节把它变成数值。做法是对 joint2 做三项单变量修改，在固定位形 q = (0.2, -1.1, 0.4, 0.1, 0) 下用同一套正解算 TCP，比较差异。固定位形是为了让差异只能归因到那一行参数，关节角一变就说不清是参数还是位形造成的。

结果见表 3-2。把 axis 从 (0,0,1) 改成 (0,0,-1)，TCP 偏移了 [-0.207, -0.042, -0.558]，模长 0.596 m，这一条最直观地说明 axis 决定的是运动方向；把 origin 的高度从 0.018 抬到 0.028，TCP 精确上移 0.010 m，误差小于 10^{-10} m，说明 origin 就是父子坐标系之间的平移；把 limit 上限从 -0.2269 收到 -0.7，正解一点没变，只是滑块行程变短，说明 limit 只约束范围、不参与运动学。三项分别对应"写错会怎样""改装配会怎样""改限位会怎样"。joint2 的父子关系、轴向与正方向示意见图 3-6。

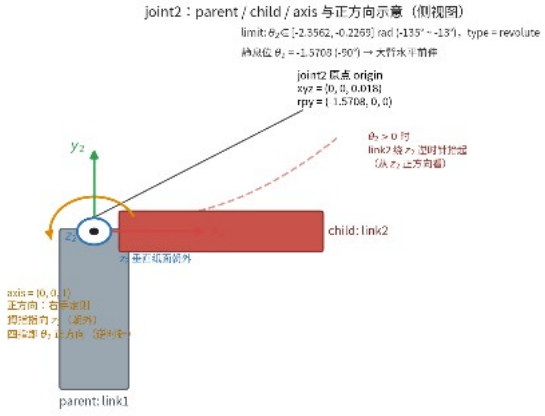


图 3-6　joint2 的 parent/child/axis 与正方向示意

表 3-2　joint2 三项参数修改实验结果

变体	修改内容	TCP 变化 Δ (m)	效果解释		
axis	joint2 旋转轴 (0,0,1) → (0,0,-1)	[-0.207, -0.042, -0.558]，\	Δ\	=0.596	旋转方向翻转，正方向与实物相反
origin	joint2 高度 0.018 → 0.028	[0, 0, +0.010]	大臂整体抬高 1 cm，TCP 沿 z 同步上移		
limit	joint2 上限 -0.2269 → -0.7	[0, 0, 0]	仅收紧限位，正解不变，滑块行程缩短		

3.10　本章小结

这一章从实物切出 13 个连杆、12 个关节，逐个讲清 URDF 的 link 与 joint 标签，给出了关节参数表、坐标系链和正运动学连乘式，用三层 launch 把模型、可视化与算法节点串成一条命令，最后用四方向核对、Gazebo 加载和三项参数修改实验，把"模型和实物一致"从主观判断变成可复现的数字。全章两条规矩：模型是唯一真相，每一步都要有证据。

模型确认之后，下一个问题是让它动起来。第 4 章从关节控制讲到逆解、轨迹与误差补偿，再往上接到颜色识别和手势指令，说明这套执行能力是怎么被感知与决策层调用的。

4　算法原理、设计与实现效果

第 3 章把模型立住了，接下来要让它动，再让它看见，最后让它听懂人的意思。这一章按这个顺序写：4.1 讲执行层的关节控制、逆解、轨迹与误差补偿；4.2 讲颜色识别与目标定位；4.3 讲识别结果怎么被组织成一次分拣任务；4.4 和 4.5 讲手势怎么被读出来、又怎么变成机械臂动作。顺序和"感知—决策—执行"相反，是因为执行层不清楚的话，视觉调出来的误差根本分不清是识别错还是到位不准。

4.1　机械臂运动控制

4.1.1　关节控制与六段式抓放

底层只有一种控制手段：给某个舵机写目标角。`set_joint_angle` 是立即写，舵机按自身设定速度转过去；`set_joint_angle_soft` 把一次大角度变化插成一系列小步，每 `TRAJECTORY_DELTA_T = 0.012 s` 写一步，运动才连续。我们在它之上定了三条规矩。单关节操作用 soft 版本；多关节协作用 `set_joint_angle_list_soft`，五个关节按同一个时间 T 规划，末端走连续轨迹，而不是"先转底座再抬大臂"的折线；joint1 单独回转时限幅 ±90°，服务端执行间隔卡在 0.1 s，防止客户端高频指令把舵机压死。

单次抓取固定成六段：接近、抓取、抬升、转移、放置、退出。这条 Z 字形路径的道理很朴素——跨区域移动前先抬到安全高度、水平挪过去、再垂直下扎，就能避开塔身、桌面和自碰撞。图 4-1 标出了三个关键高度：`SAFE_DZ = 60 mm` 是接近点相对抓取点的抬升，`LIFT_DZ = 65 mm` 是抓完抬起的高度（塔顶实测 45 mm，必须高于它），`PRE_PLACE_DZ = 70 mm` 是放置上方点。另外 `GRAB_CUBE_DZ = 5 mm` 是个小补偿，吸收方块实际高度和标称值的偏差，同时让夹爪不至于跨到桌面里。



图 4-1　Pick & Place 六段式动作与 Z 字形安全路径

代码 4-1 里有三处细节是调试换来的。第一处是先调 joint1 再移动。最初直接走笛卡尔直线，底座旋转和大臂抬升同时进行，末端在空间里走了一条弧线，抓最底层方块时蹭到桌沿；改成先把 joint1 转到方位角、再垂直下扎，蹭桌就没再出现。第二处是闭合之后先抬升再旋转。闭合后立刻旋转，方块和标定板之间会产生切向摩擦，轻则位置偏一两毫米，重则直接掉落；改成"闭合、抬升、旋转"之后，八轮分拣一次没掉过。第三处是退避时多加 15 mm，专门为最低层 P4（wz = 0）准备。

代码 4-1　六段式抓取与放置的动作封装

```
def grab_cubic(self, wx, wy, wz):
    wz += self.GRAB_CUBE_DZ                          # 抓取点高度补偿 5 mm
    self.gripper_open()
    arm_x, arm_y, arm_z = self.tf_ws2arm(wx, wy, wz)
    self.arm.set_joint_angle_soft("joint1", math.atan2(arm_y, arm_x), T=1.0)  # 先调正
轴向
    self.move2ws(wx, wy, wz + self.LIFT_UP_DZ, t=1.0)        # Approach
    self.move2ws(wx, wy, wz, t=0.4)                          # Pick
    self.gripper_close()                                     # 夹住
    self.move2ws(wx, wy, wz + self.LIFT_UP_DZ + 15, t=0.4)   # Lift + Retreat

def grab_move(self, wx, wy, wz):
    wz += self.GRAB_CUBE_DZ
    arm_x, arm_y, arm_z = self.tf_ws2arm(wx, wy, wz)
    self.arm.set_joint_angle_soft("joint1", math.atan2(arm_y, arm_x), T=1.0)
    self.move2ws(wx, wy, wz + self.LIFT_UP_DZ, t=1.0)        # Pre-Place
    self.move2ws(wx, wy, wz, t=0.4)                          # Place
    self.gripper_open()                                      # 释放
```

把六段串成航点序列就得到一章的完整任务。第 3 章的四个方块归位，按"自顶向下"把 (22,-3,68)、(25,5,90)、(30,5,45)、(3,0,0) 四个抓取点和 A 到 D 四个目标点配对；第 4 章的自动堆叠改用 `GRAB_Z = [120,75,30,0]` 加 `PLACE_POSES` 字典，主循环一个 `for` 就够，代码从四段重复压到十行。改层高或改目标位姿只动这两个列表，动作实现不用碰——这是封装最实在的收益。

4.1.2 正逆运动学与误差补偿

正解就是 3.4 节那条连乘，arm_kinematics.py 解析 URDF 拿参数。逆解用数值方法：阻尼最小二乘迭代，不收敛就做 20 次随机重启，位置门限 2×10^{-4} m。拿四角抓取点、中央堆叠各层和结束位姿共 7 个点做 IK-FK 闭环，最大位置误差 2.94×10^{-7} m。

逆解有个容易被忽略的细节是选解。同一个位置常有多组关节角，SDK 返回候选列表后取最后一个，并以当前关节角作初值，这样解出来的姿态尽量贴近当前位置，关节不会突然抡一下。这个处理在连续抓放里很关键，否则两次抓取之间机械臂会做出多余的大幅动作。

再好的轨迹也盖不住装配误差。误差主要来自三处：舵机原始角度与关节弧度的线性标定（JOINT2SERVO_K、JOINT2SERVO_B）、末端随径向距离增大而下垂、俯仰角与工作台不平行。对应两个补偿器：Z 向按径向距离线性拟合下垂量，俯仰角用平面拟合直接生成。

$$z' = z + (k\cdot r + b),\quad r = \sqrt{(x^2 + y^2)} \tag{4-1}$$

$$\text{pitch} = A\cdot\sqrt{(x^2+y^2)} + B\cdot z + C \tag{4-2}$$

PITCH_PANEL_A/B/C 分别取 −0.002490、−0.004985、3.048636，来自对工作台平面的多点标定。两项合起来，末端在任意点位的重复性能压到毫米级，12/12 和 48/48 这两个成功率就是靠它撑住的。它的局限也清楚：一阶线性模型在 wz 跨度大时不灵，从顶层 z=120 mm 到最底层 z=0，余项明显变大，所以最底层抓取额外把退避抬升加了 15 mm 兜底。用一个简单模型吃掉九成误差，剩下一成用保守参数挡住。

4.1.3 Minimum Jerk 轨迹

直接把目标角写给舵机，加速度在起停瞬间是跳变的，看起来就是起步一顿、到点一顿，夹爪里的方块跟着晃。改用 Minimum Jerk：令 $\tau = t/T$，位置取五次多项式

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5,\quad s(0)=0, s(1)=1, s'(0)=s'(1)=0, s''(0)=s''(1)=0 \tag{4-3}$$

起止速度与加速度都为零，位置从 0 平滑过渡到 1，是这类点到点运动里最柔和的方案。把 $s(\tau)$ 映射到 $[\theta_s,\ \theta_e]$，按 0.012 s 采样下发，就是关节角序

列。图 4-2 是按上式实算出来的三条曲线：位置单调上升、两端斜率为零；速度呈钟形，峰值在中点；加速度从零出发、先正后负、末端回零。加速度没有跳变，方块受的惯性冲击就小，这也是抓取动作看起来"轻"的原因。参数上统一用 T=1.0 s 做跨区域移动、T=0.4 s 做短距下扎与抬升，前者求稳、后者求节拍。

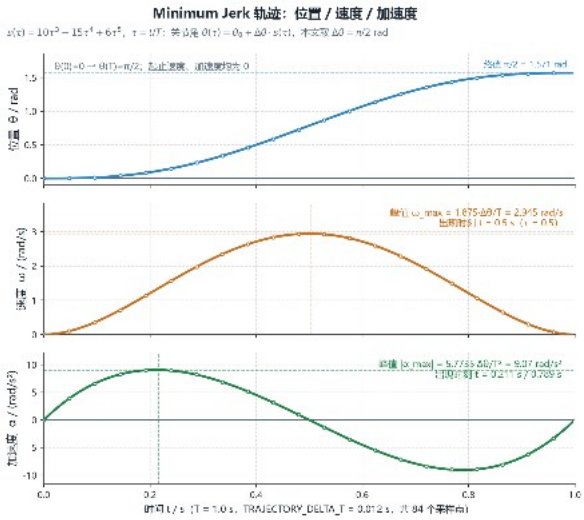


图 4-2 Minimum Jerk 轨迹的位置、速度与加速度曲线（T = 1.0 s）

4.2 HSV 颜色识别与目标定位

4.2.1 颜色空间、阈值与形态学

OpenCV 读到的是 BGR 三通道 uint8 图像。直接对 BGR 设阈值，亮度和色彩会混在一起，换个光照阈值就废了。HSV 把颜色拆成色相 H、饱和度 S、明度 V：光照变强主要抬 V，色温变化主要移 H，H 对阴影和亮度的敏感度相对低一些。OpenCV 里 H 被压到 0 到 180（色相角的一半），S 和 V 是 0 到 255。要说清一点：HSV 不是"光照不变"，S 在高光下会掉、V 在阴影里会掉，阈值还是得现场标。

阈值分割本质上是在颜色空间里圈一个长方体：`cv2.inRange` 对三个通道分别判断，只有同时落进区间的像素才置 255。红色是最典型的坑，H 在 0 和 180 附近各有一段，只设 0–10 会漏掉深红，只设 170–180 会漏掉正红，必须两段分别提取再相加。掩膜上的噪声有三类，对应两种形态学：镜头噪声形成的孤立白点用 3×3 开运算去掉，反光形成的空洞和阴影造成的边缘断裂用 5×5 闭运算补上。核尺寸不能大过目标，否则小目标会被整体腐蚀掉。

代码 4-2 红色双区间掩膜与形态学处理

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
m1 = cv2.inRange(hsv, np.array([0,   120,  70]), np.array([10, 255, 255]))   # 红 0~10
m2 = cv2.inRange(hsv, np.array([170, 120,  70]), np.array([180, 255, 255]))  # 红
```

```
170~180
mask = cv2.add(m1, m2)                          # 两段合并，缺一段就会漏检
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  np.ones((3, 3), np.uint8))   # 去孤立噪点
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, np.ones((5, 5), np.uint8))   # 填反光空洞
```

四色阈值经现场标定后落在表 4-1。备注那一列每行都对应一个踩过的坑：黄色和白纸对比度低，S 下限必须提到 100 以上才能把白纸排掉；绿色 H 跨度给到 40–85，是为了同时罩住深浅两种绿积木。

表 4-1　四色 HSV 阈值标定结果与现场备注

颜色	H 范围	S 范围	V 范围	测试效果	现场备注
红	0–10；170–180	120–255	70–255	稳定	双区间合并；抑制橙/紫误检
黄	20–35	100–255	100–255	较稳定	与白纸对比度低，S 下限敏感
绿	40–85	80–255	60–255	稳定	H 跨度大，覆盖深浅绿
蓝	96–120	157–255	100–255	稳定	采用 Notebook 默认值起步后微调

4.2.2　轮廓、中心定位与加权命中率

对每个颜色的掩膜调 `cv2.findContours`，用 `RETR_EXTERNAL` 只取外轮廓、`CHAIN_APPROX_SIMPLE` 压缩直线段；按面积降序排，把小于 50 px²（分拣场景提到 80 px²）的丢掉。对留下的轮廓算 `boundingRect`，取中心 $(u, v) = (x+w/2,\ y+h/2)$ 当目标中心。

这里有个选择值得说：为什么用边界框中心而不是矩质心。方形积木一旦被部分遮挡、或者掩膜中间有反光空洞，矩质心会被空洞拉偏；边界框中心的抗干扰性更好。代价是目标倾斜或外面伸出一块阴影时框会被撑大，所以同时用长宽比和填充率（轮廓面积除以框面积）把明显不合理的框滤掉。颜色判定用"逐色掩膜加最大轮廓"：每个颜色独立出掩膜、各取最大轮廓，如果两个颜色的框重叠（阈值交叠或反光造成），用 IoU 大于 0.55 的非极大值抑制留下面积大的那个。整条链路见图 4-3。

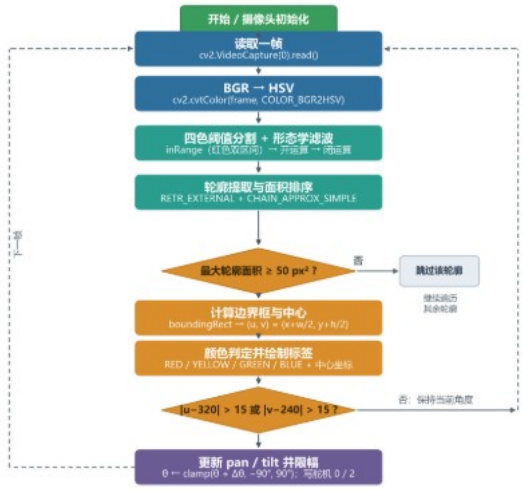


图 4-3　四色识别算法流程图

图 4-4 是四色分别标定之后的框选结果。蓝色掩膜边缘最干净；红色受反光和阴影影响会有局部缺口；黄色在浅色桌面上带少量噪点；绿色轮廓最规整。四个颜色都能在原画面里被正确框住，说明表 4-1 的阈值在当时的房间光照下是能用的，换房间就得重来，这一点第 5 章有专门的数据。

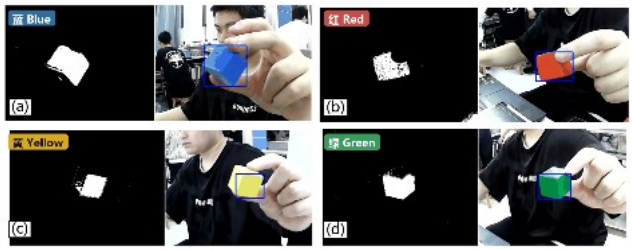


图 4-4　四色目标的识别与框选结果（蓝 / 红 / 黄 / 绿）

分拣场景的定位思路和末端相机场景不一样。实验六里相机固定俯视，方块总落在标定板中心附近，所以不必对整幅图做分割，直接取画面中心 50×50 的 ROI，转 HSV 生成四色掩膜，再用高斯权重算各色命中率取最大值。这样做有三个好处：ROI 小，单帧处理不到 10 ms；高斯加权让边缘噪声权重低；命中率本身是个置信度，可以直接和阈值 0.03 比较，而不是只看有没有像素点亮。图 4-5 用同一套阈值在现场帧上复现了这个过程：红、黄、绿的命中率分别是 0.869、0.797、0.872，蓝块落在夹爪阴影里，命中率 0.572，仍远高于阈值；画面里没有方块时四色掩膜全空、命中率 0.000，函数返回 None，系统退回观察态。这个"没东西必须返回 None"的处理看着琐碎，缺了它，空画面会被判成某种颜色，机械臂就对着空气抓一把。



图 4-5　ROI 采样、二值掩膜与四色加权命中率

4.3　自动分拣状态机

自动分拣真正难的不是识别，是保证同一个方块只被抓一次。检测线程和动作线程是并发的：动作期间画面还在刷、current_color 还在更新，动作一结束下一轮立刻又看到同一个颜色，可方块已经被搬走了，机械臂就对着空位下爪，整轮报废。解法是识别门控加颜色快照。门控用一个 detect_enabled 事件，只在 OBSERVE 态置位，确认到有效颜色后马上 clear，动作期间检测线程照常刷新画面但不再写 current_color；快照则是确认那一刻把颜色拷进 pending_color，整段动作用快照，结束再清空并把 cycle 加一。

状态是四个：INIT、OBSERVE、ACTION、IDLE；允许的转移只有五条，条件都写死：初始化完成进 OBSERVE；wait_stable_color() 在 10 s 内确认到有效颜色进 ACTION；超时或识别到映射表外的颜色就原地重新观察；投放完成且 cycle 小于 8 回 OBSERVE；cycle 等于 8 进 IDLE 并关闭识别。结构见图 4-6。



图 4-6　自动分拣状态机与视觉—动作互锁关系

代码 4-3　按帧序号去重的连续帧确认

```
CONFIRM_FRAMES, OBSERVE_DWELL, OBSERVE_TIMEOUT, CYCLE_LIMIT = 5, 2.0, 10.0, 8

def wait_stable_color(timeout):
    t0, last_color, last_seq, cnt = time.time(), None, -1, 0
    while time.time() - t0 < timeout:
        color, seq = get_color_and_seq()          # 颜色与帧序号在同一把锁内读取
        if seq == last_seq:                       # 没有新帧：跳过，避免同一帧被重复计数
            time.sleep(0.01); continue
        last_seq = seq
        if color is None:                         # 无有效色：清零，继续等到超时
            last_color, cnt = None, 0
        elif color == last_color:
            cnt += 1
            if cnt >= CONFIRM_FRAMES:
                return color
        else:
            last_color, cnt = color, 1
        time.sleep(0.01)
    return None
```

代码 4-3 是从"看着对"改到"真的对"的典型。初版每 50 ms 读一次共享变量，读到同一个颜色就加一。问题在于检测线程要是比轮询慢，同一帧会被重复计数，所谓的"连续 5 次采样"其实只有两三帧；更糟的是没有物块时计数会被 None 重置后重新累加，连续 5 次 None 直接返回，10 s 超时形同虚设。改法很简单：识别线程每帧递增 `current_seq`，确认函数按帧序号去重，只在新帧上计数，None 时清零并继续轮询到超时。改完用 6 个构造用例做单元测试，改之前只过 4 个，改之后 6 个全过。这件事的教训是：并发程序里的计数必须绑在一个单调递增的事件源上，否则"帧数"这个词本身就含糊。

4.4　手势识别

手势端跑在笔记本上（`new.py`）：`cv2.VideoCapture(0)` 取 640×480 的 BGR 帧，先 `cv2.flip(img,1)` 水平镜像（符合照镜子的直觉），转 RGB 送进 MediaPipe Hands，参数是 `max_num_hands=2`、检测与跟踪置信度都是 0.5。每帧返回最多两只手、每只 21 个关键点，外加左右手标签。程序把右手当主手、左手当副手，用 `mp_drawing.draw_landmarks` 把骨架画回预览窗口——这一笔很有用，识别是否正常，看骨架就知道，不用去猜。

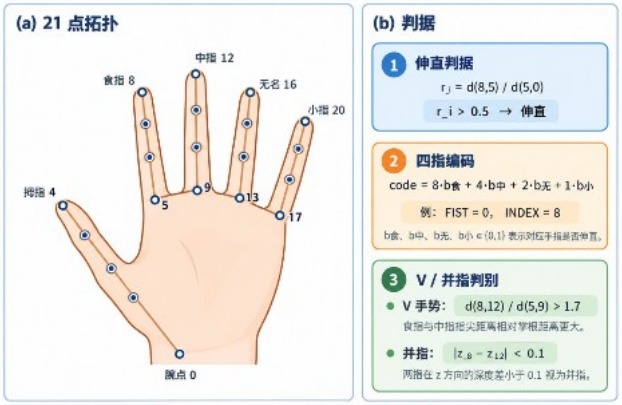


图2 MediaPipe 21点手部拓扑与手指伸直 / V / 并指判据

图 4-7　MediaPipe 21 点手部拓扑与手指伸直 / V / 并指判据

图 4-7(a) 是 21 点拓扑：0 是腕点，四指的掌指关节是 5、9、13、17，指尖是 8、12、16、20。后面所有判据都由这套编号推出来。判断一根手指是否伸直，最直接的办法是看指尖到掌指关节的欧氏距离是否大于某个阈值，但这个办法有个硬伤：手离相机近时距离大、远时距离小，固定阈值 0.5 根本站不住。所以改用带符号距离比：

$$r_i = d(\text{指尖}, \text{掌指关节}) \,/\, d(\text{掌指关节}, \text{腕点}),\quad r_i > 0.5 \text{ 判为伸直} \qquad (4\text{-}4)$$

分母给了尺度归一化，判定对手离相机的远近不敏感；带符号是为了区分指尖在掌指关节上方还是下方，手掌小角度倾斜时不至于误判。四根手指的 0/1 结果左移拼接，得到 0 到 15 的 finger 码。这个编码方式沿用自开源项目 Gesture-Controlled-Virtual-Mouse，编码保留、动作层重写（第 6 章表 6-2 有说明）。

光有手指码还不够。"食指和中指同时伸直"对应三种手形：张开成 V、两指并拢、只有中指伸出。三者手指码很接近，得再加判据。我们用两指尖距离与掌指关节距离的比值区分张开与并拢（并拢时约 1.1 到 1.3，张开时约 2.0 到 2.4，阈值取 1.7 落中间），再用两指尖的 z 差小于 0.1 区分并指与只伸中指。这两个阈值是拿本次相机和手型实测出来的，换相机就得重标，这属于规则方法的固有成本。

代码 4-4　带符号距离比、V/并指判别与 5 帧确认

```
def sign_dist(self, pts):
    d = self.dist(pts)                          # 两点欧氏距离（归一化坐标）
    return d if self.hand_res.landmark[pts[0]].y < self.hand_res.landmark[pts[1]].y
else -d

def set_finger(self):
    finger_int = 0
    for tip, mcp in [(8, 5), (12, 9), (16, 13), (20, 17)]:
        denom = self.sign_dist([mcp, 0])      # d(掌指关节, 腕点)
        if abs(denom) < 1e-6:
```

```
            finger_int = finger_int << 1; continue
        r = self.sign_dist([tip, mcp]) / denom
        finger_int = (finger_int << 1) | (1 if r > 0.5 else 0)
    self.finger = Gest(finger_int)

def get_gest(self):
    if self.hand_res is None: return Gest.PALM
    cur = self.finger
    if self.finger == Gest.FIRST2:                          # 食指 + 中指都伸直
        r = self.dist([8, 12]) / max(self.dist([5, 9]), 1e-6)
        if r > 1.7:                                         # 两指张开 → V
            cur = Gest.V_GEST
        else:                                               # 两指并拢
            z_diff = abs(self.hand_res.landmark[8].z - self.hand_res.landmark[12].z)
            cur = Gest.TWO_FINGER_CLOSED if z_diff < 0.1 else Gest.MID
    if cur == self.pre_gest: self.frame_cnt += 1
    else:                    self.frame_cnt = 0
    self.pre_gest = cur
    if self.frame_cnt > 4: self.ori_gest = cur              # 连续 5 帧一致才采纳
    return self.ori_gest
```

静态手势的稳定靠两级计数。手势级要连续 5 帧一致才被采纳，动作级要映射出的动作再连续 3 帧一致才真正下发。为什么不合并成 8 帧一次判断？因为两者确认的对象不同：前者确认用户想表达哪个手势，后者确认这个手势对应的动作没有在跳。分开计数，遇到类别边界导致动作短暂跳动时，可以靠动作计数拖一下再切，用户不用重新举手。整条流程见图 4-8，图里两个"否"分支不是异常，是常态：没检测到手就保持 hold，5 帧不一致就回候选。把"否"画成回到上游而不是结束，程序才会在下一帧继续试。

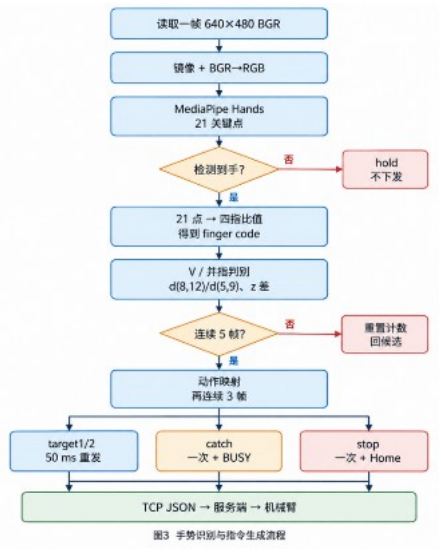


图 4-8　手势识别与指令生成的完整流程

4.5　手势指令状态机与通信

手势到动作的映射在表 4-2，三条原则：停止优先，FIST 在任何状态下都能抢占；连续动作和一次性动作分开发，target1/target2 是连续转动所以按 50 ms 周

期重发，catch 和 stop 是一次性事件所以只在动作变化时发一次；没有明确意图就不动，其余组合和无手统一映射成 hold。

表 4-2　手势—指令—动作映射与发送策略

手势类别	手指状态（食/中/无名/小）	指令	机械臂响应	发送策略
握拳 FIST	0 0 0 0（码值 0）	stop	回 Home，终止当前动作	变化时发一次，最高优先级
仅伸食指 INDEX	1 0 0 0（8）	target1	joint1 正向 +3°	50 ms 周期重发
食指+中指并拢 FIRST2	1 1 0 0（12）	target2	joint1 反向 −3°	50 ms 周期重发
食指+中指张开 V_GEST	1 1 0 0（33）	target2	joint1 反向 −3°	50 ms 周期重发
五指张开 PALM/LAST4	1 1 1 1（15）	catch	读末端位姿→世界坐标→抓取	变化时发一次，catch_busy 互斥

识别端和服务端之间只用 JSON 行通信：发一条 `{"action": "target1", "step": 3.0}\n`，服务端按换行符切流、逐行解析。这种一行一条的做法不需要长度前缀，天然能流式解析，出问题时用 nc 就能看到原始报文，半行损坏丢掉即可。服务端手里有四个状态：latest_cmd 最新指令、catch_busy 抓取互斥、stop_latched 停止锁存、pending_home 待回零。其中最新指令是覆盖写，不是队列。这个选择是刻意的：改成队列的话，机械臂会老老实实执行几秒前的意图，用户早就松手了它还在往左转。通信时序见图 4-9。

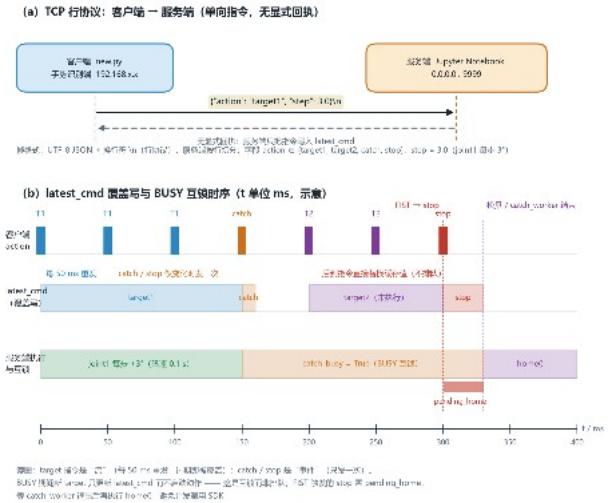


图 4-9　TCP JSON 行协议与 latest_cmd 覆盖写、catch_busy 互锁时序

代码 4-5　服务端 executor_loop：指令消费、BUSY 互锁与协作取消（节选）

```
latest_cmd = {"action": "hold", "step": 3.0}
catch_busy = False; stop_latched = False; pending_home = False
MOVE_INTERVAL = 0.1

def executor_loop():
    global catch_busy, stop_latched, pending_home, latest_cmd
    while True:
        time.sleep(0.01)                                    # 10 ms 取一次指令
        cmd = latest_cmd                                     # 覆盖写：只取最新
        if cmd["action"] == "hold":
            stop_latched = False; time.sleep(MOVE_INTERVAL); continue
        if cmd["action"] in ("target1", "target2"):
            if catch_busy:                                   # BUSY 期间不写关节
                time.sleep(MOVE_INTERVAL); continue
            delta = +3.0 if cmd["action"] == "target1" else -3.0
            pan = max(-90.0, min(90.0, pan_rad + delta))    # ±90° 限幅
            arm.set_joint_angle_soft("joint1", math.radians(pan), T=0.3)
            latest_cmd = {"action": "hold", "step": 3.0}    # 消费指令，避免重复执行
        elif cmd["action"] == "catch" and not catch_busy:
            catch_busy = True
            latest_cmd = {"action": "hold", "step": 3.0}
            threading.Thread(target=catch_worker, daemon=True).start()
        elif cmd["action"] == "stop" and not stop_latched:
            stop_latched = True
            if catch_busy:  pending_home = True             # 协作取消：等抓取线程退出
            else:           arm_app.home()
        time.sleep(MOVE_INTERVAL)                            # 0.1 s 执行间隔，防舵机被
压死
```

代码 4-5 有四个要点。最新指令是覆盖写，且每条被消费后立刻复位成 hold，这样 catch 不会因为残留再触发一次。stop_latched 保证一次 stop 只执行一次，hold 分支会把它复位，所以按住 FIST 不会反复发 home。stop 撞上 catch_busy 时只置 pending_home，等抓取线程退出时自己检查再回零，避免并发调 SDK。第四条最要紧：软急停替代不了硬件急停。grab_move 是阻塞调用时，stop 断不掉当前航点，真要紧急停止得靠 SDK 层 abort 或者直接断使能。

状态机结构见图 4-10。WAIT 是无手或 hold，SELECT 记下候选并启动 5 帧计数，CONFIRM 完成确认并映射成 action，EXECUTE 下发；target 类在 EXECUTE 里按 50 ms 重发，手一松回 WAIT；catch 进 BUSY，由 catch_busy 互斥，抓取线程收尾后回 WAIT；FIST 确认后进 STOP，BUSY 期间走协作取消，松爪后回 HOME 再回 WAIT。这里要分清互锁和排队：BUSY 期间来的新 target 只更新 latest_cmd、不启动动作，这不是排队，是被丢掉的旧意图只保留最新的那一个。

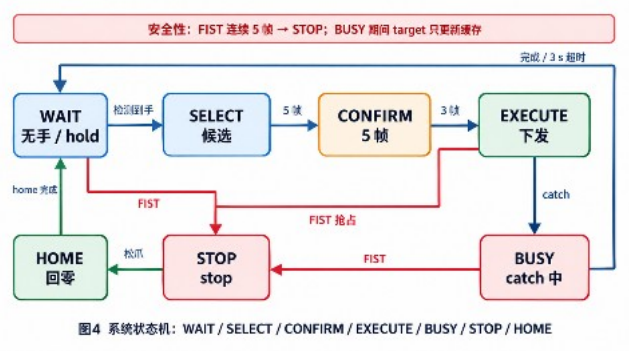


图 4-10　手势指令状态机（WAIT / SELECT / CONFIRM / EXECUTE / BUSY / STOP / HOME）

4.6　本章小结

这一章按依赖顺序讲完了三层算法。执行层用关节软运动、数值逆解、Minimum Jerk 轨迹加一阶误差补偿，把点到点定位做到毫米级，再用六段式 Z 字形路径把单次抓放封装成可复用的航点序列。感知层用 HSV 阈值加形态学加轮廓或 ROI 加权命中率，把四色识别和定位做出来，顺带说清红色要双区间、没东西要返回 None 这两个易错点；手势层用 MediaPipe 21 点、带符号距离比和 V 与并指的双判据做出五类手势，用 5 加 3 的双级确认换零误触发。决策层用两个状态机把连续输入变成离散任务，识别门控和帧序号去重挡住了重复抓取与重复计数，覆盖写和互斥挡住了指令积压与并发写关节。

程序组织上，识别端 `new.py` 里的 `ArmClient` 管连接、`HandRecog` 管单手关键点与手势码、`ArmGestureController` 管左右手分配与发送策略；服务端 Jupyter 管 `run_server`、`latest_cmd`、`executor_loop`、`catch_worker` 和 SDK 调用。两边只通过 JSON 行通信，一端换掉不影响另一端。

第 5 章要把这些算法放到实测数据里检验，包括成功率的样本量、延迟的分解，以及十二条故障各自的原因和处理。

5　综合实验结果与问题分析

第 4 章讲的是算法怎么设计，这一章讲它在真机上跑出来是什么样。所有数据来自三类记录：实验当场的录像、程序日志、离线统计脚本。凡是由其它数字推出来的，都写出推算关系，方便复核。

5.1　测试方案与指标定义

平台与第 3 章一致，这里只复述跟测试直接相关的部分：DICE 5 自由度机械臂（实验箱 6），Jetson Nano 主控，串口 `/dev/ttyTHS1`、115200-8-N-1；USB 相机 640×480、索引 0，实验五装在机械臂末端随臂运动，实验六固定在工作台正上方俯视。这两处相机装法不能同时用——两个进程都开 `VideoCapture(0)` 时后开的那个会读不到帧，所以两条通道只能分时跑，这一点在 5.5 节会再提一次。

用例分三级。单元级针对能独立判定对错的最小对象，比如 URDF 参数变体、IK-FK 闭环、颜色确认函数，要求是"错了能定位到哪一行"；集成级把模块串成任务，比如六段式抓放、3 轮归位、3 轮堆叠、8 块分拣、10 次手势，考查接口通不通；回归级用固定样本把同一组用例重跑一遍，确认修改没有引入新问题。之所以不拿"演示一遍成功"当证据，是因为这类动作耗时且不可逆，掉一次块、碰一次桶，场景就变了，单次成功说明不了可复现。

六项指标刻意分开"能力"和"节拍"。成功率按块次算；定位误差取末端 TCP 实测位姿与目标位姿的欧氏距离；单轮节拍从上一轮复位完成算到下一轮观察位就绪，不含人工补料（5.3 节另有一组含人工补料的整轮节拍，两组口径不同）；识别率按保持段统计；误触发率是"无指令场景下错误下发次数 / 测试次数"；端到端延迟取手势首次稳定出现到舵机开始转动的时间差。用例与判据见表 5-1。

表 5-1　集成与回归测试用例及验收判据

层级	用例	输入与操作	通过判据	样本量
集成	六段式 Pick & Place	3 轮、每轮 4 块归位	12/12 成功，定位误差 ±2 mm	12 块次
集成	自动堆叠主循环	3 轮、每轮 16 个动作	48/48 成功，平均单步 2.4 s	48 动作
集成	乱序颜色分拣	8 块随机供料	8/8 正确，零重复抓取	8 块
集成	手势—指令—动作	10 次随机手势	10/10 正确，0 误触发	10 次

层级	用例	输入与操作	通过判据	样本量
回归	离线帧分类	录像按 2 fps 抽帧	134/134 判对，最低命中率 0.243	134 帧
回归	稳态手势识别	5 类手势、每类 180 保持帧	每类识别率 100%	900 帧
回归	端到端延迟	10 次触发计时	0.38~0.47 s，均值 0.42 s	10 次

5.2 建模与运动控制的实测结果

模型验证没有只截一张图。RViz 里依次切正视、侧视、俯视、斜视，同时把 TF 坐标轴打开（Marker Scale 0.25），逐项对着表 3-1 读：正视图看 joint1 是否竖直、整机是否在同一水平面回转；侧视图看 joint2 水平轴、静息位 -1.5708 rad 时大臂是否水平前伸；俯视图确认料场布局和 x/y 方向；斜视看夹爪的 mimic 联动。四个方向都没查出父子接反或轴向错误，这条结论很重要——后面的逆解、抓取位姿、相机投影全部建立在这棵树上。

Gazebo 那边验证的是同一份 URDF 的物理可用性。empty_world 给地面和光照，spawn_model 用 `-urdf -param robot_description` 加载，带 `-J joint2 -1.5708` 预设静息位，模型站稳、不穿地、不发散；再用 `gz joint` 的位置 PID 把 joint1 拧到 ±90° 环绕一圈，腰关节正方向和实物一致。参数修改实验把"看懂字段"变成了数字：axis 反向让 TCP 偏 0.596 m，origin 抬 0.01 m 让 TCP 精确上移 0.010 m，limit 收紧则正解不变。模型上 7 点 IK-FK 闭环最大误差 2.94×10^{-7} m，远小于 2×10^{-4} m 门限，说明后面测到的 ±2 mm 不是运动学的锅。

六段式抓放跑下来很稳。接近段从抓取点上方 60 mm 垂直下探，抓取段用 `GRAB_CUBE_DZ = 5 mm` 补偿块高，抬升段上抬 65 mm 高过塔顶，转移段先转 joint1 对正方位再直线横移，放置段降到目标顶面张开夹爪，退出段再抬 65 mm。第 3 章的四块归位做了 3 轮、每轮 4 块，12/12 全部成功，定位误差 ±2 mm，其中稳态到位和抬升 50 mm 后的动态跟随误差都在 ±2 mm 内，放置后各点偏差在 ±1.5 mm 内。第 4 章的自动堆叠按"自顶向下、层层剥离"做，3 轮共 48 个动作全过，平均单步 2.4 s。

节拍上有个看得见的改进：整轮耗时从 13 s 压到 9.6 s，降幅约 26%（由 (13-9.6)/13 推算）。拆开看，约 4 s 花在 4 次抓取抬升，约 5 s 花在 4 次横移

放置，剩下 0.6 s 是复位与等待。省下来的时间来自三处改动：一是减少 home 回退，把"先回 home 再横移"改成"就地抬升、转 joint1、直线横移"；二是分段时间分配，接近与横移用 T=1.0 s，下扎与放置用 T=0.4 s；三是按层动态抬升 50/55/60/65 mm，低层不过抬、高层不欠抬。这三处动的是动作序列的结构，不是把速度调快，所以成功率没掉。

这几个 100% 要说清楚条件：同一工位、固定供料、固定光照、标准方块，而且调试阶段针对低层碰桌和横移干涉各改过一轮参数，12/12 与 48/48 里还留着"夹爪反馈超差就重试两次"的机制。±2 mm 也只在重复性意义上成立，换台面或重新装配后必须重标 `z_error_polyfit` 与 `PITCH_PANEL`。

5.3 颜色识别与自动分拣

识别流程是：每帧按宽 600 px 缩放，在画面中心取 50×50 的 ROI，转 HSV 后按四色阈值生成掩膜，高斯加权算各色命中率取最大，低于 0.03 就判为无目标。现场帧上复现的结果是红 0.869、黄 0.797、绿 0.872，蓝块落在夹爪阴影里只有 0.572，仍远高于阈值；画面里没有方块时四色掩膜全空、命中率 0，函数返回 None 并退回观察态。

乱序分拣 8 块全对，红入 B1、黄入 B2、绿入 B3、蓝入 B4，没有出现同一块被抓两次。图 5-1 是投放段的实拍时序，原片 t=69 到 72 s 共四帧，红块被送进左前方的红桶；抓取段（t=65 到 68 s 的下探、闭合、抬升、回转）截图没有放进正文，录像里可以逐帧查到。单轮节拍 9.5 到 24.8 s、均值 17.5 s，跨度大是因为中间有人工补料。把人工环节剔掉，单轮五段是：回观察位 0.8 到 1.2 s、停留观察 2.0 s、5 帧确认 0.2 到 0.3 s、抓取（含 home 往返）约 2.4 s、旋转投放约 2.6 s，合计 8.0 到 8.5 s。这组数字说明时间几乎全花在机械臂往返和投放上，识别单帧不到 10 ms，提节拍该去缩行程，而不是去优化图像处理。

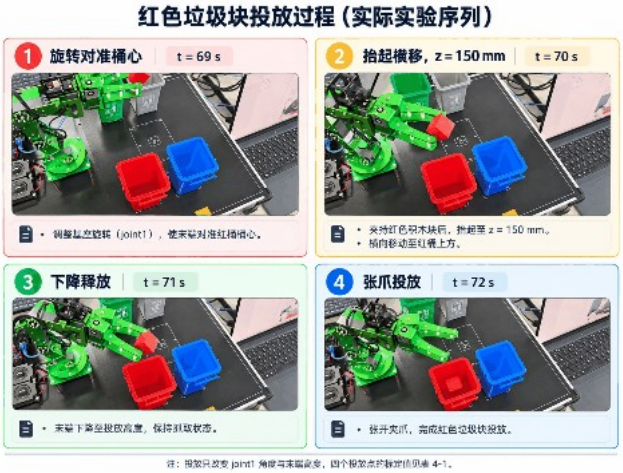


图 5-1　红色物块抓取投放实测时序（抓取段与投放段）

单次演示的说服力有限，所以又在录像上做了离线复现：按 2 fps 抽 134 帧（红 42、黄 45、绿 23、蓝 24），用与程序完全相同的阈值分类，134 帧全部判对。逐色平均命中率是红 0.988、黄 0.824、绿 0.741、蓝 1.000，最低的 0.243 仍然是判定阈值 0.03 的 8.1 倍；次优色得分的中位数是 0.000、最大 0.287。按独立样本近似，134/134 的双侧 95% Wilson 置信下限约 0.972，但连续帧之间可能相关，这个区间只能当参考。

这里必须交代方法边界。程序里的 ROI 是画面中心的固定窗口，这个前提来自"相机固定俯视加标定板"；录像却是手持拍摄、标定板逐帧漂移，固定窗口用不了，所以复现时先用已知目标颜色的掩膜定位物块再取 ROI。也就是说，134 帧全对衡量的是"ROI 已经对准之后"的分类稳定性，不能当成端到端检出率。为了说明这个差别，又做了一次对照：把定位换成颜色中立的方式（取任意高饱和像素的重心）后，140 帧里只有 32 帧判对，占 22.9%，绿色甚至是 0/23，因为 ROI 多半落在高饱和的绿色机械臂上。两组样本量不同，不该直接比准确率，但对照指出的问题很清楚：ROI 定位才是误差的大头。真机上靠固定相机和固定观察位把这一环锁死；要在任意视角下工作，就必须引入独立的定位环节，比如模板匹配、轮廓检测或手眼标定。

5.4　手势识别与人机交互搬运

手势通道定义了五类有效手势：握拳 FIST、只伸食指 INDEX、食指中指并拢 FIRST2、食指中指张开 V_GEST、五指张开 PALM（码值 15，也叫 LAST4）。手指伸直用带符号距离比判（比值大于 0.5 算伸直），V 与并指再用 d(8,12)/d(5,9) 大于 1.7 和两指尖 z 差小于 0.1 区分。

识别率按每类 300 帧的片段统计，片段含进入、保持、退出三段，只把中间的保持段（180 帧/类）计入，5 类共 900 帧全部判对。这统计的是稳态识别率，不是全过程逐帧准确率，进入和退出段本来就会有过渡帧。左右手分类置信度均值在 0.97 到 0.99 之间，最低 0.92；判定裕量（四指比值与 0.5 的最小绝对差）是 FIST 0.31、INDEX 0.26、FIRST2 0.24、V_GEST 0.19、PALM 0.22（逐帧统计见实验记录，口径与 4.4 节一致）。V 的裕量最小，跟它最容易和并指混淆的现象对得上——二维图像里两指并拢和张开，指尖距离只差十几个像素。FIST 裕量最大，因为四指全弯时每根手指的比值都离 0.5 很远。



图 5-2　四种手势的识别结果（FIST / INDEX / V_GEST / LAST4）

随机手势测试做了 10 次，全部正确触发，0 次误触发，五类手势各覆盖两次。图 5-3 是一次 catch 指令下的动作序列：初始位张开，识别到 PALM 后下探到木块上方，闭合后抬起，最后回待机位。执行端由状态机管着 WAIT、SELECT、CONFIRM、EXECUTE、BUSY、STOP、HOME 七个状态，候选手势连续 5 帧一致才进动作映射，动作再连续 3 帧一致才下发；catch 进 BUSY 并由 `catch_busy` 互斥，FIST 可协作取消，BUSY 期间新来的 target 只更新 `latest_cmd`。

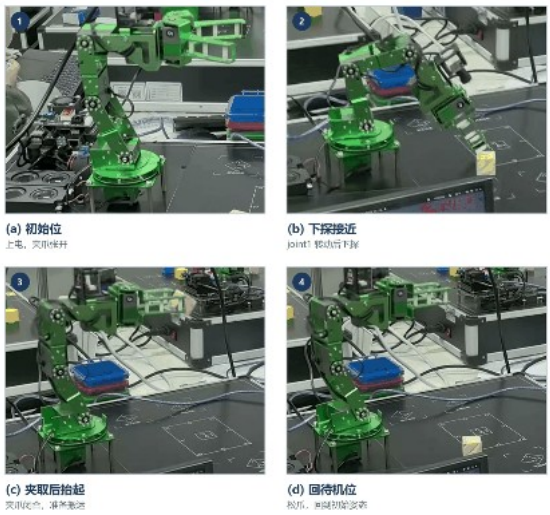


图 5-3　手势触发机械臂动作序列：初始位→下探→夹取后抬起→回待机位

端到端延迟拆成六段，见表 5-2，典型值合计约 0.40 s，10 次实测落在 0.38 到 0.47 s，均值 0.42 s，和分解对得上。这张表给出的结论有点反直觉：延迟的大头是两级确认窗口合计 267 ms，不是网络也不是推理——MediaPipe 只占 18 ms，TCP 只占 15 ms。系统"慢"几乎全部来自我们主动设的确定性策略。把 5 帧降到 3 帧按 30 fps 推算能省 67 ms 左右，但边界帧更容易被采纳，误触发风险上去；最后还是选 5 加 3，优先保演示的确定性。真要压到 0.2 s 量级，得改确认策略（比如按裕量自适应决定帧数）并缩短舵机启动时间，换模型不解决问题。

表 5-2　手势指令到机械臂动作的端到端延迟分解

环节	典型耗时（ms）	说明
图像采集与预处理	8	读帧、镜像、BGR 转 RGB
MediaPipe 推理	18	21 点与左右手标签，CPU 推理未加速
5 帧手势确认	167	5/30 fps，主要延迟项
3 帧动作确认	100	防止类别边界导致动作跳变
TCP 与服务端调度	15	JSON 行发送与 latest_cmd 更新
舵机启动响应	90	软运动启动加机械间隙
合计	约 400	实测 0.38~0.47 s，均值 0.42 s

5.5　端到端综合演示

综合演示把两条通道按时间串起来：先用颜色通道走完识别、确认、抓取、投放、复位，再让手势通道接管 joint1 的增量转动和 catch。图 5-4 的六帧取自原始录像的原片时间 t=5.0、22.0、25.0、45.0、51.0、55.0 s，覆盖待机、识别、抓取

抬升、搬运、投放、回收六个阶段。帧与帧之间的大段时间差主要是人工补料与等待，把各段动作耗时相加，一轮净动作约 30 s——两个时间轴说的不是同一件事，不能相减得到动作时长。从时间分布看，观察与确认占了近三成，其中停留 2.0 s 是为了让自动曝光稳定下来；抓取与投放几乎压不动，因为舵机速度和最小安全抬升量都已经贴到下限。

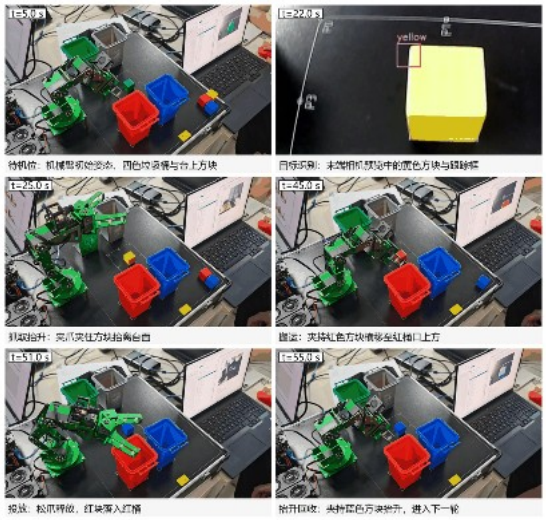


图 5-4　演示视频关键帧总览（原片 t=5~55 s，六个阶段）

两条通道共存有三条绕不开的约束。相机：两个进程都用 `cv2.VideoCapture(0)` 时后开者会失败或读到空帧，所以不能同时采集，只能分时跑，由 launch 参数决定本次启用哪一路。关节：target 指令和视觉伺服都要写 joint1，同时下发会互相覆盖，现在靠 BUSY 互锁按"谁执行谁占 joint1"仲裁，属于时间上的互斥，不是空间上的融合。时序：两路独立采集、没有统一时间戳，抓取点可能用到旧帧。所以双通道融合对我们来说是扩展设计，不是已完成的功能——手势链路验过了，颜色中心暂时只作为 catch 定位的候选输入，手眼标定和统一时间戳还在计划里。

5.6　算法优缺点

六个模块各有各的适用边界。HSV 静态阈值可解释、能现场标定、单帧不到 10 ms，判错了看一眼掩膜就知道是 H、S 还是 V 的问题；它的短处是阈值只在当前光照下成立，自动曝光和高光会让同一块颜色落进不同区间。面积加边界框中心比矩质心更抗掩膜空洞和局部遮挡；但只给轴对齐框，目标一倾斜中心就偏，像素到毫米的换算完全依赖固定相机。双级帧确认实现简单、延迟确定；代价是固定帧数不适应动作速度，267 ms 的延迟全由它贡献。纯几何手势判据不用训练、CPU 上 18 ms、每

条判据都能解释；但它假设手掌大致正对相机，转大了带符号距离的符号可能翻，而且只支持静态手势。覆盖写 `latest_cmd` 天然丢弃过期意图；缺陷是没有执行回执，也表达不了"先 A 后 B"。状态机互锁给共享资源划了所有权；但它挡不住阻塞的 SDK 调用，也替代不了硬件急停。

把六个模块摆在一起看，主要误差出在模块交界处：定位依赖相机几何，抓取依赖标定，互锁依赖线程时序。下一节的故障表里，被归为"设计缺陷"的几条全部集中在共享状态与时序，被归为"参数问题"的则集中在阈值与高度，这个分布和上面的判断互相印证。

5.7 故障、原因与处理

表 5-3 从各次实验的"问题—原因—修改—验证"记录里挑了 12 条，挑选标准是：现象能用录像或日志复现、修改后有同组复跑数据、问题对最终指标有影响。

表 5-3 故障—原因—修改—验证汇总

序号	模块	现象	根因	措施	验证结果
1	建模	joint2 正方向与实物相反	axis 误设为 (0,0,-1)	改回 (0,0,1)，变体脚本断言 TCP	axis 变体 Δ=0.596 m 被捕获
2	建模	catkin_make 无法在工作区编译	U 盘为 exFAT，不支持符号链接	复制到 ~/arm2025_ws/src 再编译	编译通过
3	建模	TF 时有时无，同名节点顶号	旧 roslaunch 未完全退出	演示前 rosnode list 检查并清理	启动后 TF 稳定
4	堆叠	最低层 P4 抓取时夹爪碰桌	抓取点距桌面仅 5 mm，自重与舵机延迟	先对正 joint1，退避抬升额外加 15 mm	3 轮 P4 均一次成功
5	堆叠	横移时剐蹭已堆叠的方块塔	统一 50 mm 抬升量不足	按层动态抬升 50/55/60/65 mm	3 轮未再推动塔身
6	视觉	正红色块偶发漏检返回 None	H 跨 0 边界，单区间覆盖不到 175–180	双区间 [0,8] 与 [170,180] 并集	红块命中率 0.869，8 轮全对
7	视觉	黄块放白底上被判为无物体	白纸反光抬高 V、拉低 S	黄 S 下限提到 50、V 提到 100、H 收到 15–40	命中率 0.797，未再返回 None
8	分拣	同一物块被连续抓取两次	检测线程与状态机共享 current_color	加 detect_enabled 门控，确认后快照	8 轮零重复抓取
9	分拣	无物块时 10 s 超时形同虚设	把 5 次 50 ms 采样当成 5 帧	每帧递增序号并按帧号去重，None 清零轮询	6 个用例单元测试 6/6（修复前 4/6）

序号	模块	现象	根因	措施	验证结果
10	手势	食指与中指并拢被误判为 V	两类均为两指伸直，缺指间张开判据	加 d(8,12)/d(5,9)>1.7 与 z 差<0.1	10 次测试无错判
11	手势	连续 target 以 20 Hz 下发，舵机抖	客户端频率高于服务端执行能力	客户端 50 ms 周期、服务端 0.1 s 间隔、±90° 限幅	运动连续，无小步抖动
12	手势	catch 重复触发，抓取被新指令打断	每帧都发 catch，没有互斥	只在动作变化时发一次，catch_busy 互斥	一次 catch 只抓一次

这十二条里有三条不属于"参数没调好"，而是设计缺陷。第 9 条的本意是连续 5 个不同帧给出同一结论，可实现上每 50 ms 读一次共享变量，识别线程一慢，同一帧就被重复计数，极端情况下 5 次采样等于一次瞬时判定；无物块时更糟，连续 5 次 None 就提前返回，10 s 超时白设。加上帧序号去重之后，同一组 6 个单元用例从 4/6 变成 6/6，说明它不可能靠调参解决。第 8 条是共享 current_color 没有所有权边界，动作期间识别线程还在改颜色，"这轮抓什么"取决于动作结束那一瞬间变量是什么值，重复抓取是迟早的事。第 12 条是每帧都发 catch，等于把一次性事件当电平信号用。三条的共同点是共享状态缺少时序约束，或者事件与电平的语义混了，属于并发与接口设计层面。

其余各条更接近工作点标定或环境适配。HSV 阈值、抬升高度这类是工作点，换光照换台面就得重标；exFAT 不支持符号链接、端口 TIME_WAIT、路径写死成 /home/ubuntu 属于工程化不足，跟算法无关但决定演示能不能一次跑通。把两类分开记，复盘时才知道哪些改动该写进代码，哪些写进操作手册就够了。

图 5-5 把表 5-3 这 12 条按模块和根因重新统计：建模 3 条、手势 3 条最多，堆叠、视觉、分拣各 2 条；按根因分，阈值与工作点 5 条、状态与时序 4 条、环境与工具链 2 条、标定与装配 1 条，合计 12 条。这个分布说明两件事：问题并不都集中在"视觉算法难"，建模和装配阶段同样吃掉大量时间；真正难缠的是参数标定和并发时序，前者靠现场标定加锁定曝光，后者只能靠把状态边界设计清楚。

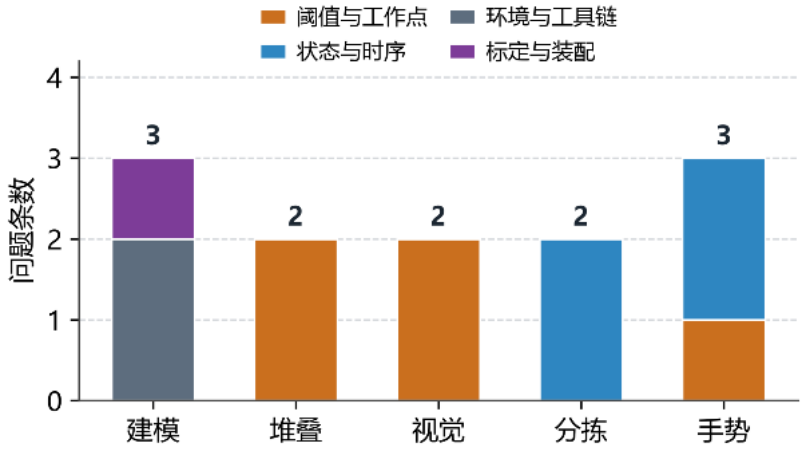


图 5-5　表 5-3 的 12 条问题分布（按模块与根因类别）

5.8　课程思政体会

资源权衡在这个项目里是被算力逼出来的，不是纸面上的概念。Jetson Nano 的 128 核 Maxwell GPU 我们没启用（OpenCV 与 MediaPipe 的 Python 路径都走 CPU，手势推理还放在笔记本上），MediaPipe 单帧就占 18 ms，如果把颜色识别也换成深度学习模型，识别线程和显示线程会互相抢 CPU，延迟与帧率一起恶化。所以颜色通道选 HSV 阈值加几何规则：单帧不到 10 ms、行为可解释、参数能现场用滑块标定，不需要标注数据也不需要模型部署，省下的算力留给控制与交互。这不是不会用深度学习，是在延迟、可解释性和开发成本之间做了一次取舍。开源复用同样有边界：手势编码的 0 到 15 位思路借鉴了开源项目 Gesture-Controlled-Virtual-Mouse，但六段式抓取序列、TCP JSON 行协议、七状态状态机、BUSY 互锁与协作取消全部由本组重写，因为上游只解决"手势到鼠标事件"，没有机械臂的离散动作和安全约束。

工程安全是被事故教出来的。关节角限幅 ±90°，防止 target 指令连续累加越过机械限位；服务端加 0.1 s 执行间隔，防止客户端 20 Hz 的高频指令把舵机压死；catch 用 `catch_busy` 互斥，保证抓取过程不被重入；stop 做成协作取消，FIST 确认后置 `pending_home`，由抓取线程在安全时机回零，而不是并发调 SDK。有一条边界必须说明白：软急停替代不了硬件急停，`grab_move` 阻塞时 stop 断不掉当前航点。另一次教训来自投放路径：从灰桶回观察位时腕部刚到绿桶，桶被推移，查下来是 z=150 mm 到 z=15 mm 之间做笛卡尔直线插补，路径正好扫过绿桶上方；改成观察位与抓取位分离、每轮结束先高位复位、投放点取桶心并留 8 mm 余量之后，后续 4 轮没再剐到。这些措施没一条是事先想到的，全是看现象反推出来的。

工匠精神体现在把"看着能用"变成"可复现、可量化"。134 帧离线复现是最典型的一次：现场 8/8 全对的时候，我们仍然把录像按 2 fps 抽帧、用同一套阈值重跑分类，正是这次复现把"ROI 已对准"和"端到端检出"之间的差距揪了出来，也才有了 140 帧、22.9%、绿色 0/23 的对照实验。颜色确认函数的 6 个单元用例也是主动补的，补之前只过 4 个，说明当时所谓的"确认"根本没覆盖无物块、同帧重复这些边界。四方向截图、参数变体断言、录像帧号、每轮完成日志都留了档，目的是让结论能被第三人复核，这比多写几句"效果良好"有用。

团队协作方面，最大的体会是"模块单独通过不等于集成通过"。三个人各管一条线：建模与运动控制、视觉与分拣、手势与通信，每周固定一次联调。真正的时序 bug 是在互相 review 时被点出来的：管手势的同学复核分拣 Notebook，发现确认函数以 50 ms 轮询共享变量却从不检查帧号；管分拣的同学复核手势服务端，发现 catch 每帧都发且没有互斥。这两处都属于接口契约没写清——`latest_cmd` 到底是队列还是覆盖写、BUSY 的所有权归识别端还是服务端，必须在动手写代码之前用文字定下来。

5.9 改进方向

识别层优先级最高，因为它是误差链条的入口。短期可以把固定阈值换成 HSV 直方图加 KNN：对每个颜色采样若干帧的 H/S 直方图当模板，在线做最近邻分类，能吃掉光照的整体漂移；或者改用 Lab、YCbCr 空间，把亮度与色度分得更开。工程上应该加自动曝光与白平衡锁定，让阈值工作点在整场演示里保持不动；更关键的是把 ROI 定位从"固定窗口"升级成独立环节——先用模板匹配或轮廓检测找到标定板和物块，再取局部 ROI，这样 22.9% 才不会成为致命伤。

决策层的方向是把"示教"换成"测量"。现在桶位靠一次性示教，桶被碰一下就全乱；如果能完成手眼标定，把像素坐标经相机、末端、基座三级变换映射到基座坐标，再由视觉伺服把末端对准桶口，就能适应桶位漂移，也才谈得上真正的双通道融合。系统层可以补三件事：夹爪加电流或在位检测，把开环序列变成"抓—检—投"闭环；用 MoveIt 做路径规划与避障，替代笛卡尔直线插补，从根上避免剐蹭；把手部 21 点序列送进 LSTM 或 1D-CNN 做时序分类，支持挥手、捏合这类动态手势，同时把确认延迟压下来。改进顺序按"对指标的影响与实现成本"排：曝光锁定与 ROI 独

立定位成本最低、收益最直接；手眼标定与视觉伺服次之；MoveIt 和动态手势成本最高，适合留给后续课程设计。

5.10 本章小结

这一章用统一口径给出了四个子系统的实测结果：12/12 块次归位、48/48 堆叠动作、8/8 乱序分拣且零重复抓取、10/10 手势触发且零误触发、五类手势共 900 帧保持段识别率 100%（每类采集 300 帧，计入 180 帧），端到端延迟均值 0.42 s，其中 67% 花在两级确认上而不是推理。除了数字，更想强调三点：把"100%"按样本量分级看（134/134 和 12/12 的说服力不一样）、把"ROI 已对准"和"端到端检出"分开评价、把故障分成设计缺陷和工作点两类。这些结论直接支撑第 6 章的分工说明和后续改进的判断。

6 组员分工与参考文献

6.1 分工

本组三人。分工按"每人一条能独立验收的技术线，外加一项交叉职责"来划，目的是让每个人都能被单独提问，也都能指出自己写过的文件。张航宁管建模与运动控制这条公共底座，张浩轩管手势识别与通信，吴锐超管视觉分拣与系统集成。三条线在第 4 章那个"识别—决策—搬运"的闭环处汇合，接口和联调由三个人共同承担。具体分工见表 6-1。

表 6-1 组员分工、贡献度与可被提问的技术点

姓名	学号	负责模块	对应代码文件	贡献度
张航宁（组长）	24061733	URDF 建模与 RViz/Gazebo 验证；机械臂运动控制与 Pick & Place	urdf/ arm5dof.urdf、arm5dof_uservo.py 、arm_test.py	40%
张浩轩	24061720	手势识别（MediaPipe 与手指判据）；TCP 通信与指令服务端	new.py、我的通讯识别.ipynb	32%
吴锐超	24061734	HSV 颜色识别与自动分拣；状态机互锁与系统集成测试	我的垃圾分类.ipynb、14.OpenCV 物体追踪.ipynb、config.py	28%

划分依据是依赖方向，不是谁有空。URDF 与运动控制是三个人都要调用的底座，必须先稳；手势和颜色是两条并列的输入线，可以并行开工；集成测试单独成岗，负责把两条线放到同一台机器上跑通。这么排的好处是任何一条线返工时，另外两条还能继续跑自己的单元测试；代价是集成阶段要额外约定接口，这部分在 6.2 节说。

6.2 协作方式与贡献度

代码用 git 管，主干 main，三个人分别在 feature/urdf、feature/gesture、feature/color 三条分支上写，合并前必须由另一个人 review。每周一次周会，只议三件事：本周跑通了哪些单元测试、跨模块接口有没有变、下周联调哪个场景。联

调日志按"时间、模块、现象、临时处理、是否已修复"五列记，事后复盘能直接对上表 5-3。

互相 review 确实捞到过自己看不见的问题。张浩轩复核分拣的确认函数时指出，`wait_stable_color()` 以 50 ms 轮询共享变量却从不检查帧号，识别线程一慢同一帧就会被重复计数，"连续 5 帧"退化成"连续 5 次采样"，这就是表 5-3 里第 9 条；吴锐超复核手势服务端时指出 catch 每帧都发、没有互斥，抓取过程可能被新指令打断，后来补了"只在动作变化时发送一次"和 `catch_busy`。两次得到同一个教训：接口的时序语义要在写代码之前用文字说清，不能等联调时再猜。

贡献度按代码行、提交次数、联调时长三项折算，结果是 40%、32%、28%（见表 6-1）。差异主要来自职责范围，不是工作量绝对值：建模、SDK 二次封装和两套抓放流程都在张航宁这一侧，代码量最大；张浩轩的代码集中在两个文件里，但排障耗时多；吴锐超复用了部分上游思路、自研量相对小，不过集成测试和全流程回归是他扛的。三人都至少参与过一次完整演示的现场操作与录制。

6.3 参考文献

以下文献按 GB/T 7714—2015 著录。

[1] Open Robotics. URDF tutorials: learning URDF step by step[EB/OL]. [2026-10-18]. https://wiki.ros.org/urdf/Tutorials.

[2] OpenCV. Color conversions: OpenCV documentation[EB/OL]. [2026-10-18]. https://docs.opencv.org/4.x/de/d25/imgproc_color_conversions.html.

[3] ZHANG F, BAZAREVSKY V, VAKUNOV A, et al. MediaPipe Hands: on-device real-time hand tracking[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops. Seattle: IEEE, 2020.

[4] HOGAN N. An organizing principle for a class of voluntary movements[J]. The Journal of Neuroscience, 1984, 4(11): 2745-2754.

[5] CRAIG J J. Introduction to robotics: mechanics and control[M]. 3rd ed. Upper Saddle River: Pearson Prentice Hall, 2005.

[6] QUIGLEY M, CONLEY K, GERKEY B, et al. ROS: an open-source Robot Operating System[C]//ICRA Workshop on Open Source Software. Kobe: IEEE, 2009.

[7] MACENSKI S, FOOTE T, GERKEY B, et al. Robot Operating System 2: design, architecture, and uses in the wild[J]. Science Robotics, 2022, 7(66): eabm6074.

[8] TSAI R Y, LENZ R K. A new technique for fully autonomous and efficient 3D robotics hand/eye calibration[J]. IEEE Transactions on Robotics and Automation, 1989, 5(3): 345-358.

[9] 蔡自兴. 机器人学[M]. 3 版. 北京: 清华大学出版社, 2015.

[10] 冈萨雷斯. 数字图像处理[M]. 4 版. 阮秋琦, 译. 北京: 电子工业出版社, 2020.

[11] 全国信息与文献标准化技术委员会. 信息与文献 参考文献著录规则: GB/T 7714—2015[S]. 北京: 中国标准出版社, 2015.

[12] SICILIANO B, KHATIB O. Springer handbook of robotics[M]. 2nd ed. Berlin: Springer, 2016.

这些文献在正文里的落点：[1][6][7] 对应第 3 章的 URDF 标签、launch 结构与节点通信；[4][5][9][12] 对应第 4 章的轨迹规划与运动学；[2][3][8][10] 对应第 4、5 章的 HSV 分割、手势关键点、手眼标定和改进方向；[11] 是全文格式依据。

6.4 开源仓库与外部资源

本组项目已经整理成公开仓库：https://github.com/zhang-hn-1/Robot-Final，课程代码 (2026-2027-1)-S0610350-04。仓库 README 写了运行环境（Jetson Nano、Ubuntu 18.04、ROS1、Python 3）、编译与启动步骤（`catkin_make`、`roslaunch display.launch`、`roslaunch stack.launch`）、三位组员各自负责的模块，`urdf/`、`launch/`、`rviz/`、`src/` 分别放模型、启动文件、可视化配置与算法节点。

外部资源按任务书要求标注来源，并写清借鉴了什么、自己改了什么，见表 6-2。

表 6-2　外部开源资源引用与自研边界

资源	来源网址	借鉴内容	自研与修改
Gesture-Controlled-Virtual-Mouse	https://github.com/Viral-Doshi/Gesture-Controlled-Virtual-Mouse	四指状态位组成 0~15 手势码的思路	距离比判据、V 与并指判别、5+3 双级确认、状态机与 TCP 协议全部重写
MediaPipe Hands	https://google.github.io/mediapipe/solutions/hands.html	21 点手部拓扑与左右手分类接口	直接调用 Python API，未修改模型；手势规则与主副手分配自研
OpenCV	https://opencv.org	颜色空间转换、阈值分割、形态学与轮廓函数	四色阈值、红色双区间、ROI 加权命中率与帧序号去重自研
DICE 5DoF 机械臂 SDK	https://www.dice.com.cn/	串口舵机协议、关节角下发与末端位姿接口	软运动封装、z 误差补偿、六段式抓放与状态机自研
ROS 与 RViz/Gazebo	https://www.ros.org	URDF 描述规范、TF 发布与可视化、物理仿真	本机械臂 13 连杆 12 关节模型与 launch 结构自研

没有直接复制过完整开源项目。上游仓库只用来理解手势编码的组织方式和调用接口，动作层、通信层、安全互锁全部按本机械臂的控制特性重写；引用到的第三方库保留原始许可证与出处，README 里一并列了。

7 总结与展望

7.1 技术收获

- **ROS 与 URDF**：模型的正确性可以用数值证明——改 axis 让 TCP 偏移 0.596 m、改 origin 精确上移 0.010 m、改 limit 不影响正解，三组对照把标签语义从"背下来"变成"可测量"。
- **运动学**：解析正解与阻尼最小二乘逆解在 7 个目标点上闭环误差 2.94×10^{-7} m；实机 ±2 mm 的误差来自标定、舵机间隙与装配，不在运动学层。
- **视觉**：HSV 阈值只在固定相机、固定观察位、锁定曝光的前提下才是稳定判据；ROI 定位一旦偏离设计假设，统计会从 134 帧全对掉到 140 帧仅 32 帧判对。
- **状态机**：把连续识别离散化为事件的关键是帧号与互斥，而不是帧数本身；把"5 次采样"当成"5 帧"会让确认机制整体失效。
- **通信与交互**：覆盖写指令适合"只执行最新意图"的交互，但没有执行回执、也不能表达"先 A 后 B"，多步任务需要状态机而不是队列。

7.2 方法论收获

模型即真相，每一步有证据。坐标系、抓取点与相机投影都从同一份 URDF 推导，模型一错后面全错，所以建模阶段值得做四方向核对与变体断言；四方向截图、参数变体脚本、录像帧号与每轮完成日志构成从"现象"到"结论"的链条，报告里每个数字都能追溯到某张图或某行日志。

把连续识别离散化为事件，用数据而非感觉判断稳定性。识别线程每帧都在输出，机械臂动作却离散且耗时，两者之间必须有确认、快照、互斥与状态机。单次跑通不叫稳定，3 轮、8 块、134 帧、10 次动作才形成结论，且要用样本量校正"100%"——12/12 与 134/134 的可信程度并不相同。

7.3 不足与后续方向

第一，**双通道尚未真正融合**：共用一台相机只能分时采集，共用 joint1 只能靠 BUSY 互斥仲裁，两路图像没有统一时间戳；后续需完成手眼标定，用视觉伺服替代固定桶位的示教。第二，**视觉与手势都工作在"标定好的工位"上**：阈值与判据都是特

定光照、相机与手型下的工作点，换环境就要重标；后续可用 HSV 直方图加 KNN 提高光照适应性，并把 ROI 定位升级为独立的模板匹配或轮廓检测环节。第三，**安全与自检仍不完整**：系统只有软件互锁，没有硬件急停回路，夹爪也没有电流或在位检测；后续应补独立急停与关节速度限制，用"抓—检—投"闭环替代开环序列，并接入 MoveIt 做路径规划与避障。

7.4 对本次课程的意见与建议

其一，希望把"排错记录"纳入过程性评价。本课程最耗时的不是写代码，而是查"为什么 TF 时有时无""为什么同一物块被抓两次"这类问题；这些问题在课堂上往往一句带过，却是真正的工程能力。若课程能要求每组提交故障日志（现象、复现步骤、定位过程、最终原因），并在验收时抽问其中一条，学生对系统的理解会比只看演示深刻得多。

其二，建议提供一层可切换的仿真验证环境。实体实验箱的排队与硬件故障会显著压缩调试时间；若能先在 Gazebo 或 RViz 中用同一份 URDF 跑通运动学与状态机，再上实机验证标定与舵机响应，本组"模型先行、实机收尾"的路线会更容易推广。

其三，建议给小组项目一份"最小接口清单"模板。本次三人协作最大的体会是"接口先行"比"任务均分"更重要：只要话题表与关节接口先定死，三个人可以并行开发；反之即使每人代码量相同，联调阶段仍会互相阻塞。

8 AI 工具使用说明

按照课程任务书对 AIGC 内容的要求，本章逐项说明本组在哪些环节使用了 AI 工具、生成了什么内容、以及人工做了哪些核对与修改。

使用范围共有五类：一是**报告文字润色与结构梳理**，在已有实验记录与章节提纲基础上调整语句、统一术语，章节结构与技术判断由本组确定；二是**文献检索辅助**，查找 URDF、OpenCV、MediaPipe Hands、最小加加速度轨迹、手眼标定等方向的经典文献条目，再逐条到官方文档或数据库核对；三是**示意图与统计图的绘制脚本**，matplotlib 绘图代码与配色排版由 AI 协助编写，图内数据全部来自本组实验记录、日志与录像统计；四是**Word 排版脚本辅助**，标题层级、图表题注、三线表与页码等脚本由 AI 协助生成，再按课程格式要求逐项校改；五是**排障思路的头脑风暴**，作为排查线索，最终是否成立以实验复现为准。概括而言，本报告的事实、数据与结论由本组人工确定，报告文字是在人工给出事实与结论的前提下用 AI 做润色与结构整理，示意图由 matplotlib 脚本绘制且图内数据取自实测，Word 版由排版脚本生成。

必须明确声明：全部实验数据、运行截图、演示视频与核心算法代码（URDF 模型、运动控制与六段式抓取、HSV 识别与分拣、手势识别与状态机）均为本组在机械臂实验平台上实际完成、实际调试、实际录制；AI 未生成任何实验效果图，未虚构任何实验数据，也未生成可直接运行的全套代码。报告中出现的成功率、节拍、命中率、延迟与误差等数字，均可在代码包内的日志、录像与离线统计脚本中找到来源。

人工修改完善举三例。其一，AI 初稿中的通用描述被替换为本组真实参数与实测值，例如"通过多次测试验证了识别稳定性"被改写为"离线复现 134 帧全部判对、逐色平均命中率 0.988/0.824/0.741/1.000、最低 0.243 是阈值 0.03 的 8.1 倍"，并补上"ROI 已对准"的方法边界。其二，删除 AI 初稿中列出而本组实验中并未出现的功能（如"基于深度学习的动态手势识别""自动避障路径规划"），只作为第 5 章的改进方向保留。其三，把英文术语统一为课程与代码中的说法（hand landmark→手部关键点、frame debounce→帧确认、servo jitter→舵机抖动），joint 编号统一为 joint1~joint5。

本组对 AI 辅助生成的全部文字、代码与图表脚本均做了人工核对：事实逐条比对实验记录，参数逐个对照配置文件，引用逐篇查证来源；凡与实验记录不符之处以本组记录为准。若本说明与代码包内日志、录像不一致，以代码包内日志与视频为准。

9　其他附件说明

本组交付物除报告正文外，还包含演示视频、原始录像、代码包与素材清单，如表 9-1 所示。

表 9-1　附件清单

编号	文件名	内容	位置
1	第 6 组_张航宁_张浩轩_吴锐超_综合设计报告.docx / .pdf	正文第 1~9 章、图表与参考文献	01_综合设计报告/
2	第 6 组_演示视频_机械臂目标识别与手势搬运.mp4	建模可视化、六段式抓取、颜色分拣、手势搬运（2 分 23 秒）	02_演示视频/
3	实验原始录像（mp4 若干）	未剪辑素材，含分拣与手势联调过程	02_演示视频/原始素材/
4	第 6 组_张航宁_张浩轩_吴锐超_代码包.zip	README、src、urdf、launch、rviz 与自检文档	03_项目代码包/
5	00_提交说明_材料清单.md	目录结构、命名规则与提交前核对清单	交付根目录/
6	图源清单（真实截图与示意图）及绘图脚本	逐张图片的来源与可复跑脚本	04_图源与素材清单/

演示视频不超过 3 分钟，按"建模与可视化—运动控制与堆叠—颜色分拣—手势搬运"四段组织，关键节点配口述说明系统功能与当前状态；视频为 1280×720、30 fps，与实验录像同源，便于逐帧核对。代码包复现步骤为：在 Jetson Nano（Ubuntu 18.04）或 PC 上安装 ROS1、Python 3、OpenCV、NumPy 与 MediaPipe；把功能包复制进工作空间后执行 `catkin_make` 并 `source devel/setup.bash`；用 `roslaunch` 依次加载模型、场景与算法节点；运行时确认串口为 `/dev/ttyTHS1`（PC 端为 `COM6`）、相机索引为 0。README 中另附三人各自负责的代码模块，便于按模块检查。